



MAY UNIVERSITY IN CAIRO
FACULTY OF ENGINEERING

Computer Engineering Program

Swarm Emergency Robotics System

Autonomous Multi-Agent Coordination for Disaster Response

Graduation Project

Spring 2026

Prepared By:

Marwan Ramadan Ahmed
Mahmoud Mohamed Matouk
Mahmoud Naser Abdelkareem
Mohamed Ahmed Tawfik
Yehia Mostafa Gaber

Abdulrahman Ahmed Fathy
Mohamed Esmael Mahmoud
Momen Mohamed Sharkawi
Hamed Ahmed Nabih

Supervised By:

Dr. Hassan Ibrahim

Committee Report

We certify we have read this graduation project report as examining committee, examine the student in its content and that in our opinion it its adequate as a project document for “ ”.

Supervisors:

Name:

Signature:

Date: / /202

Examiners:

Name:

Signature:

Date: / /202

Intellectual Property Right Declaration

This is to declare that the work under the supervision of Dr. Hassan Ibrahim having title Swarm Emergency Robotics System carried out in partial fulfillment of the requirements of Bachelor of Engineering at Faculty of Engineering is the sole property of May University In Cairo and the respective supervisor. It is protected under the intellectual property right laws and conventions. It can only be considered/ used for purposes like extension for further enhancement, product development, adoption for commercial/organizational usage, etc. with the permission of the University and respective supervisor. This above statement applies to all students and faculty members.

Names:

Supervisor:

Acknowledgement

We would like to express our deepest gratitude to Dr. Hassan Ibrahim for his invaluable guidance, continuous support, and expert supervision throughout the development of this Swarm Robot System project. His technical insights and constructive feedback have been instrumental in shaping the direction and quality of this research.

We extend our sincere thanks to the Department of Computer Engineering at May University in Cairo for providing us with the necessary resources and support required for the successful completion of this project.

We also acknowledge the challenges we faced as Computer Engineering students learning new domains such as electronics, mechanical design, and robotics hardware. The support from our colleagues and the broader robotics community through online resources and documentation was invaluable.

Finally, we would like to thank our families for their encouragement and understanding throughout the duration of this demanding academic endeavor.

Abstract

The Swarm Robot System represents an approach to autonomous collaborative robotics, designed for environments such as search and rescue operations. This project presents a framework for coordinating multiple autonomous robots working together to accomplish tasks.

The system architecture is built upon three specialized cooperative robotic platforms. The Alpha robot is responsible for LiDAR-based SLAM using `slam_toolbox` to generate precise 2D environmental maps. The Beta robot hosts the camera for YOLO-based fire, smoke, and flame detection, and carries pump and servo mechanisms for fire intervention. The Gamma robot is an ESP32-based unit responsible for gas sensing and environmental monitoring using an MQ-5 sensor.

A key aspect of our approach is the implementation of a coordinated architecture where all three robots contribute to a shared map representation. The software framework is built on the Robot Operating System 2 (ROS 2), with each Raspberry Pi-based robot running ROS 2 locally. Communication between the Pi-based robots and the operations center uses ZeroMQ (ZMQ) with `msgpack`-serialized envelopes over a 5 GHz WiFi network, while the ESP32-based Gamma robot communicates via HTTP/JSON endpoints.

This project also represents significant learning for the team, as Computer Engineering students with limited prior exposure to electronics, mechanical design, and robotics hardware. The team divided responsibilities into programming, design, electronics, and environment preparation, while maintaining cross-functional understanding.

The system currently supports autonomous mapping and object detection within the mapped environment. The testing environment is a 4x4 meter area divided into four rooms.

Keywords: Swarm Robotics, Multi-Robot Systems, SLAM, Sensor Fusion, Object Detection, Search and Rescue, ROS, YOLO

Table of Contents

Acknowledgement

Abstract

Table of contents

List of figures

List of tables

List of abbreviations and acronyms

Chapter 1: Introduction

1.1 Project Overview.....	12
1.2 Background and Context.....	13
1.3 Problem Statement.....	15
1.4 Motivation.....	16
1.5 Project Objectives	17
1.6 Project Scope	18
1.7 General Constraints.....	19
1.8 Team Organization.....	20
1.9 Organization of the Dissertation.....	20

Chapter 2: Literature Review and Related Work

2.1 Introduction to Swarm and Cooperative Robotics	23
2.2 Review of Multi-Robot and Swarm Systems	23
2.2.1 Fundamental Concepts.....	24
2.2.2 Reviewed Projects and Systems	25
2.3 Review of Search and Rescue Robotic Systems.....	25
2.3.1 Existing SAR Robot Systems	26
2.3.2 Key Features Identified	26
2.4 Review of SLAM Techniques.....	26
2.4.1 SLAM Fundamentals.....	26
2.4.2 SLAM Algorithm Review.....	27
2.4.3 Sensor Technologies for SLAM	28
2.4.4 Sensor Fusion	28
2.5 Review of Object Detection in Robotics	28
2.5.1 Deep Learning for Object Detection	28
2.5.2 YOLO (You Only Look Once).....	29
2.5.3 Deployment Considerations.....	29
2.6 Review of Communication Frameworks.....	30
2.6.1 Robot Operating System (ROS)	30
2.6.2 Network Communication.....	30
2.6.3 Dashboard and Monitoring	31
2.7 Review of Hardware Components	31
2.7.1 Processing Platforms.....	32
2.7.2 Sensors	32

2.7.3 Motors and Drive Systems	33
2.7.4 Power Systems	33
2.8 Review of Mechanical Design Approaches	33
2.8.1 Chassis Design	33
2.8.2 Design Workflow	34
2.9 Summary of Literature Review	34
Chapter 3: System Analysis and Design	
3.1 Project Planning and Feasibility	37
3.1.1 Development Methodology	37
3.1.2 Project Timeline and Gantt Chart	39
3.2 Problem Analysis and Need for the System	40
3.3 Requirements and Users	41
3.3.1 User and Stakeholder Description	41
3.3.2 Functional Requirements	41
3.3.3 Non-Functional Requirements	42
3.4 System Architecture	43
3.5 Hardware and Software Design	45
3.5.1 Hardware Design	45
3.5.2 Software Design	48
3.6 Communication and Dashboard Design	52
3.7 Risks, Assumptions, and Dependencies	55
Chapter 4: Implementation	
4.1 Hardware Implementation	59
4.2 Software Implementation	62
4.2.1 SLAM Implementation	62
4.2.2 Object and Human Detection Implementation	63
4.2.3 Sensor Integration	64
4.2.4 Robot Communication Implementation	64
4.2.5 Dashboard Implementation	65
4.3 System Integration	66
Chapter 5: Testing, Results, and Evaluation	
5.1 Testing Strategy and Environment	70
5.1.1 Testing Methodology	70
5.1.2 Physical Test Environment	70
5.1.3 Hardware and Network Setup	71
5.2 Test Cases and Metrics	71
5.2.1 SLAM and Mapping Metrics	71
5.2.2 Visual Perception Metrics	71
5.2.3 Sensor Fusion and Localization Metrics	72
5.2.4 Gas Detection and Network Communication Metrics	72
5.3 Experimental Results	72
5.3.1 SLAM and Mapping Results	72
5.3.2 Visual Perception Results	74
5.3.3 Sensor Fusion and Localization Results	74

5.3.4 Environmental Sensing Results	75
5.3.5 Network and Communication Results	76
5.4 Analysis and Discussion	76
5.4.1 System Strengths and Achievements	76
5.4.2 Limitations and Constraints	77

Chapter 6: Conclusion and Future Work

6.1 Conclusion ..	
6.2 Achievements of the Project	
6.3 Future Enhancements	
6.4 Potential Real-World Deployment	

List Of Figures

Figure 2.1 – Main hardware components	31
Figure 3.1 – Multi-robot system configuration	34
Figure 3.2 – Multi-Robot System Architecture and Communication Framework.....	43
Figure 3.3 – CAD model of the robot chassis	45
Figure 3.4 – Final Assembled Design of the Mapping Explorer Robot	45
Figure 3.5 – Software Stack and ROS Node Architecture	47
Figure 3.6 – Native PySide6 desktop monitoring dashboard	51
Figure 3.7 – Real-Time Dashboard Interface	52

List of tables

Table 1.1: Application domains of swarm robotic systems	14
Table 2.1: Classification of swarm robotics coordination strategies	22
Table 2.2: Comparison of multi-robot path planning approaches	23
Table 2.3: Summary of SLAM techniques used in mobile robots	26
Table 2.4: Comparison of object detection models used in robotic perception	28
Table 3.1: Main components and specifications of the Mapping Explorer Robot	45
Table 3.2: Specifications of sensors used in the robotic system	47
Table 3.3: Software technology stack used in the robotic system	50
Table 4.1: Power Architecture Components and Specifications	59
Table 5.1: SLAM and Mapping Performance Results	73
Table 5.2: YOLOv8n Object Detection Results	74
Table 5.3: Localization and Odometry Errors	74
Table 5.4: MQ-5 Gas Sensor Performance	75
Table 5.5: Network Communication Metrics	76

List OF Abbreviation

Acronym	Full Form
AI	Artificial Intelligence
CNN	Convolutional Neural Network
EKF	Extended Kalman Filter
GPS	Global Positioning System
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
ML	Machine Learning
PWM	Pulse Width Modulation
ROS	Robot Operating System
SAR	Search and Rescue
SLAM	Simultaneous Localization and Mapping
TCP/IP	Transmission Control Protocol/Internet Protocol
WiFi	Wireless Fidelity
YOLO	You Only Look Once

Chapter 1

Chapter 1: Introduction

1.1 Project Overview

The Swarm Robot System project represents an advancement in multi-robot autonomous systems design, specifically targeting collaborative robotics for environmental exploration and hazard assessment. This project presents a framework for coordinating multiple autonomous robots working together to accomplish tasks that would be difficult or dangerous for a single robotic unit[1][2].

The fundamental concept underlying this project is the principle of “Collaborative Intelligence” – distributing computational and sensory tasks across multiple specialized robotic units, each contributing specific functions while working in coordination. Our system deploys three specialized robots that together handle all required functions: environmental mapping using SLAM technology, fire detection using deep learning, gas sensing for hazard monitoring, and fire intervention capabilities.[1], [3]

- **Project Context and Significance**

In contemporary applications such as search and rescue operations, disaster assessment, and structural inspection, the deployment of human personnel is often dangerous and time-consuming. The Swarm Robot System addresses these challenges by providing an autonomous platform that can systematically explore unknown environments, identify hazards, locate objects of interest, and generate actionable intelligence in real-time.

The innovation in our approach lies in architecting an integrated system where each robot's capabilities are purposefully designed, and where data from all robots feeds into a shared, unified representation of the environment. This shared map becomes the foundation upon which all robots make decisions, enabling coordinated behavior[4][1].

- **System Composition**

The system comprises three autonomous mobile robots that work together to perform all required functions:

1. **Alpha Robot (Mapping and Navigation):** Equipped with an RPLidar A1 LiDAR sensor and advanced onboard processing using Raspberry Pi 4 (8 GB RAM). This robot performs continuous SLAM using `slam_toolbox` to generate precise 2D environmental maps, with `rf2o` laser odometry providing motion estimation. Alpha's wheel encoders are present but unused for localization; mapping relies entirely on scan-based odometry.[4]
2. **Beta Robot (Intervention and Fire Detection):** Equipped with Raspberry Pi 3B+, a USB camera, GY-87/MPU6050 IMU, four 25GA370 encoder motors (408 CPR), HC-SR04 ultrasonic sensors, a water pump, and a servo mechanism. This robot hosts the camera for YOLO-based fire detection, carries fire intervention tools (pump and servo), and uses custom complementary encoder and gyroscope odometry fusion for localization and navigation.[5]
3. **Gamma Robot (Environmental Monitoring):** Equipped with an ESP32 DevKit microcontroller, an MQ-5 gas sensor, HC-SR04 ultrasonic sensor, MPU6050/GY-87 IMU, a buzzer, and a servo mechanism. This robot is responsible for detecting

4. hazardous gases and monitoring environmental conditions. Gamma communicates via HTTP/JSON endpoints and supports over-the-air (OTA) firmware updates. Unlike the Pi-based robots, Gamma does not run ROS and operates independently with its own embedded HTTP interface.[5]

All robots communicate over a 5 GHz WiFi network. The Raspberry Pi-based robots (Alpha and Beta) run ROS 2 locally and expose telemetry, map data, video, and commands through a ZeroMQ/msgpack gateway protocol. The ESP32-based Gamma robot uses HTTP/JSON endpoints for telemetry and control. ROS 2 DDS traffic is deliberately confined to each robot's local network and is not transmitted over WiFi.[6]

• Technical Approach

Key technical aspects of this project include:

- **Shared Map Architecture:** The Alpha robot generates an occupancy grid map using LiDAR-based SLAM with `slam_toolbox`. The map is published through the ZMQ gateway and received by the PySide6 operations center, which maintains a shared state store for all connected robots.[5]
- **Sensor Fusion:** Alpha uses `rf2o` laser odometry combined with `slam_toolbox` for localization. Beta uses a custom complementary filter that fuses wheel encoder data with gyroscope readings, including a slip gate to reject unreliable measurements.
- **Deep Learning Integration:** YOLO-based fire, smoke, and flame detection runs in a crash-isolated child process on the operations center laptop, with detections projected onto the shared map as incident markers.
- **Real-time Dashboard:** A native PySide6 desktop operations center provides dockable panels for map visualization, fleet status, live video with AI overlays, joystick/WASD teleop, fire and gas alerts, diagnostics, and mission planning with A* route planning.[6]

1.2 Background and Context

• Project Background and Prior Research

Before beginning the implementation of this project, extensive research was conducted to understand existing work in the field of multi-robot systems, SLAM technology, and search and rescue robotics. This research phase was critical as the team members, being Computer Engineering students, had limited prior exposure to robotics hardware, electronics, and mechanical design[7].

The research phase involved reviewing numerous academic papers, open-source projects, and existing implementations of similar systems. Key areas of study included:

- Previous implementations of mapping robots and SLAM algorithms
- Multi-robot coordination frameworks and communication protocols
- Object detection systems using deep learning
- ROS (Robot Operating System) architecture and packages
- Sensor integration techniques for mobile robots

This review of prior work provided the foundation for understanding what components would be needed, what challenges to expect, and what approaches had been successful in similar projects[8].

• Evolution of Robotic Systems

The field of robotics has undergone significant transformation over the past two decades. Early robotic systems were primarily stationary, task-specific machines designed for manufacturing environments. The emergence of mobile robotics introduced new challenges: navigation in unstructured environments, real-time decision making, sensor uncertainty management, and power-constrained operation[1].

The evolution toward multi-robot systems represents the next step in robotics. Rather than solving increasingly complex problems with increasingly complex individual robots, the research community has recognized the advantages of distributed systems: redundancy, parallelization of tasks, natural scalability, and fault tolerance.

• Search and Rescue Applications

Search and rescue (SAR) operations represent one of the most compelling use cases for autonomous robotic systems. Traditional SAR operations are characterized by:

- Environmental Hazards: Unstable structures, toxic atmospheres, extreme temperatures
- Information Scarcity: Lack of prior maps or detailed environmental knowledge
- Time Pressure: Every hour of delay reduces survival probability
- Resource Constraints: Limited availability of trained personnel and specialized equipment

Robotic systems can address these challenges by providing rapid, systematic environmental assessment without exposing human personnel to hazards[1], [4].

A summary of the major application domains of swarm robotic systems, along with representative tasks and their advantages, is presented in **Table 1.1**.

Table 1.1: Application domains of swarm robotic systems

Application Domain	Example Tasks	Advantages
Search and rescue	Victim detection, mapping	Fast area coverage
Environmental monitoring	Pollution detection	Distributed sensing
Military	Surveillance	Robust coordination
Agriculture	Crop inspection	Scalability
Industrial automation	Warehouse logistics	Efficiency

• SLAM Technology

Simultaneous Localization and Mapping (SLAM) has evolved from a theoretical concept to a practical, reliable technology. Key developments in SLAM include particle filter-based approaches (FastSLAM), visual SLAM systems (ORB-SLAM), and integration with deep learning. Our implementation uses `slam_toolbox`, a modern and actively maintained SLAM framework within the ROS 2 ecosystem, combined with `rf2o` laser odometry for scan-based motion estimation on the mapping robot.[5][9]

- **Deep Learning for Object Detection**

The integration of deep learning with robotic systems has transformed robot perception. Object detection systems such as YOLO enable robots to recognize visual hazards with high accuracy. Our project uses YOLO primarily for fire, smoke, and flame detection within the mapped environment, with the detection system supporting selectable models through the PySide6 operations center.[10][11]

- **Robot Operating System (ROS)**

ROS has emerged as the standard platform for robotics research and development. ROS provides:

- Standardized Middleware: Publish-subscribe message passing and service calls.
- Sensor Integration Libraries: Support for various sensors via dedicated drivers.
- Algorithm Libraries: Implementations of SLAM, motion planning, and control algorithms.
- Simulation Environment: Gazebo simulator for testing without physical robots.
- Community and Ecosystem: Extensive documentation and pre-built components[6].

- **Learning Challenges**

As Computer Engineering students, the team faced significant learning challenges during this project. Prior to this project, the team had limited exposure to:

- Electronics and circuit design
- Robotics hardware and mechanical systems
- 3D design and CAD software (such as Inventor)
- Metal fabrication and chassis construction

This required substantial time investment in learning these skills from the ground up, which added to the complexity and timeline of the project. Each team member working on a specific domain still maintained general understanding of other parts of the project to ensure effective collaboration.

1.3 Problem Statement

- **The Challenge of Autonomous Multi-Robot Systems**

Despite significant advances in individual robot capabilities, deploying multiple robots to work collaboratively in unknown environments presents several challenges [2]:

- **Challenge 1: Environmental Representation**

Problem: Different robots need to work within the same physical environment. Without a shared representation of that environment (a map), robots cannot coordinate movements or target specific locations. Traditional approaches require each robot to independently map its environment, which creates consistency problems when different robots observe the same space.

Impact: Redundant computation, lack of true coordination, potential conflicts when different robots' maps disagree about environmental features.

- **Challenge 2: Localization Without Global Reference**

Problem: Most autonomous systems rely on GPS for localization. GPS is unavailable indoors, underground, or in urban environments – precisely the environments where search and rescue

operations occur. Without GPS, robots must localize themselves using only onboard sensors. Traditional odometry accumulates error over distance, making it unreliable for larger environments.

Impact: Robots cannot reliably navigate, cannot return to locations of interest, and task completion becomes unreliable.

- **Challenge 3: Data Processing in Real-Time**

Problem: Modern object detection systems require significant computational resources that exceed the capabilities of low-power embedded systems. Running these algorithms on a Raspberry Pi results in slow performance. Transmitting raw video streams from robots to an external processing station creates bandwidth demands.

Impact: Slow response times, inefficient bandwidth utilization, potential communication delays.

- **Challenge 4: System Reliability**

Problem: Any component failure (sensor malfunction, processor crash, network connectivity loss) can affect the system if not properly handled. Operating robots in challenging environments means component issues are likely.

Impact: Potential mission interruption, need for robust error handling. The implemented system addresses these concerns through acknowledged command delivery with retry and deduplication logic, a multi-layer deadman safety chain from console to gateway to firmware, per-stream freshness monitoring, and an emergency stop mechanism that propagates across the entire system.

- **Challenge 5: Real-Time Coordination**

Problem: Coordinating multiple robots requires reliable communication and data synchronization. Network latency and message loss can affect coordination. The challenge is finding the right balance between centralized monitoring and distributed operation.

Impact: Coordination difficulties, vulnerability to communication issues.

- **Combined Problem Formulation**

The overarching problem addressed by this project is: How can we design a reliable multi-robot system capable of autonomously exploring unknown environments and generating actionable intelligence through coordinated sensor deployment and information sharing?

This problem encompasses:

- Environmental representation and map sharing
- Reliable localization in GPS-denied environments
- Real-time distributed intelligence
- System reliability and communication
- Practical implementation using available components

1.4 Motivation

- **Operational Motivations**

The motivation for this project originates from real-world operational needs:

Search and Rescue Operations: When buildings collapse, become flooded, or are exposed to dangerous conditions, rescue teams face difficult choices. Robotic systems can compress the timeline of initial assessment. Our system specifically addresses the need for rapid environmental mapping combined with search for objects of interest and identification of hazards.

Educational Motivation: For Computer Engineering students, this project provides hands-on experience across multiple domains: embedded systems programming, distributed computing, artificial intelligence, sensor systems, and systems engineering. It bridges the gap between theoretical concepts taught in university courses and practical implementation challenges.

Research Contribution: While SLAM, sensor fusion, and object detection are individually well-studied, their integration into a practical multi-robot platform requires novel engineering work. Our approach of coordinating three specialized robots with shared mapping and distributed sensing demonstrates practical multi-robot coordination.[1][2]

• **Technical Motivations**

Sensor Fusion: The integration of multiple odometry sources—including wheel encoders, gyroscope data, and laser scan matching—demonstrates the power of multi-sensor integration. This technique improves localization accuracy compared to using odometry alone.

Edge Computing: Deploying deep learning inference while maintaining responsiveness on low-power robots addresses a practical challenge in robotics: how to execute computationally complex algorithms within power and latency constraints.

ROS Standardization: Building the system within the ROS ecosystem ensures that results are reproducible and extensible. Each component (SLAM, object detection, communication) is implemented as modular ROS packages[12],[13].

• **Learning Motivation**

A significant motivation for this project was the opportunity to learn new skills outside the traditional Computer Engineering curriculum:

- **Programming:** Learning ROS, YOLO, and robotics-specific programming paradigms.
- **Electronics:** Understanding circuits, motor control, sensor integration.
- **Mechanical Design:** Learning 3D design using Inventor and metal fabrication.
- **System Integration:** Combining hardware and software into a working system[11].

1.5 Project Objectives

• **Primary Objectives**

The project is organized around clearly defined objectives[6]:

• **Objective 1: Design and Implementation of Robot Platforms**

Develop three specialized robot platforms that work together to perform all required functions:

- **Alpha Robot (Mapper):** SLAM capability using slam_toolbox, environmental mapping with LiDAR, rf2o laser odometry
- **Beta Robot (Intervener):** Camera integration, YOLO-based fire detection, fire intervention via pump and servo, encoder-based navigation
- **Gamma Robot (Inspector):** Gas detection using MQ-5 sensor, environmental monitoring via ESP32 HTTP telemetry, hazard alerting

Success Criteria:

- Robots achieve autonomous navigation in structured indoor environments
- Robots can generate environmental maps using SLAM
- YOLO-based detection identifies fire, smoke, and flame hazards, with general object detection available as an alternative model configuration
- Gas sensor detects hazardous gases and provides real-time environmental data

- **Objective 2: Shared Map Architecture**

Implement a map representation that enables coordination:

- Develop mechanisms for generating environmental maps from LIDAR data
- Implement ZMQ gateway-based map sharing between robots and the operations center
- Validate that robots can localize on shared maps

- **Objective 3: Sensor Fusion and Localization**

Implement sensor fusion algorithms for accurate localization, using rf2o laser odometry for the mapping robot and custom complementary encoder-gyroscope fusion for the intervention robot:

- Fusion algorithms improve accuracy compared to single-sensor odometry alone
- System maintains localization during continuous operation

- **Objective 4: Object Detection and Classification**

Integrate deep learning models for visual hazard detection:

- Deploy YOLO for object detection
- Detect fire, smoke, and flame hazards in the environment, with selectable models for general detection
- Link detected objects to locations on the shared map

- **Objective 5: Multi-Robot Communication**

Establish reliable communication infrastructure:

- Implement ZMQ/msgpack gateway protocol for Pi-based robots and HTTP/JSON for the ESP32-based robot
- Use ZMQ over 5 GHz WiFi for structured, reliable data transmission with acknowledged commands
- Enable monitoring and control from central dashboard

- **Objective 6: User Interface and Monitoring**

Develop dashboard for operators to monitor and command the system:

- Display robot positions on shared map
- Show camera feed and detections
- Provide control interface for robot operation

- **Secondary Objectives**

- **Objective 7: Test Environment**

Create a test environment for demonstrating robot capabilities:

- Build 4x4 meter test area divided into 4 rooms
- Provide realistic indoor environment for testing

- **Objective 8: Documentation**

Provide documentation for the system:

- Design documentation of hardware and software
- Implementation guides
- Operational documentation for using the system

1.6 Project Scope

- **Hardware Scope:**

- Design and assembly of three specialized mobile robot platforms (Alpha, Beta, Gamma)
- Integration of sensors (LIDAR, IMU, encoders, camera, ultrasonic sensors)
- Power management and battery systems
- Motor control and drive electronics
- Chassis design and fabrication

- **Software Scope:**

- ROS-based software architecture
- SLAM implementation using slam_toolbox with rf2o laser odometry
- Sensor fusion algorithms for localization
- Deep learning pipeline for object detection using YOLO
- Communication using ZMQ/msgpack protocol for Pi robots and HTTP/JSON for ESP32
- Dashboard interface for monitoring and control

- **Environment Scope:**

- Testing conducted in custom 4x4 meter indoor test environment
- Environment divided into 4 rooms for testing
- Structured environment with walls and obstacles

- **Out of Scope:**

- Outdoor GPS-based navigation
- Aerial robots or drones
- Advanced manipulation capabilities
- Autonomous decision-making and action execution (current phase focuses on mapping and detection)
- Real-time path planning for moving obstacles

- **Assumptions**

The project makes the following assumptions:

1. WiFi network (5GHz) will be available in the operating area
2. Operating environment is structured indoor space
3. Environment is relatively static during operation
4. Operator is available to monitor the system

1.7 General Constraints

- **Hardware Constraints**
- **Power Constraints:**
 - Battery capacity limits continuous operation time
 - Charging infrastructure required between operations
- **Processing Constraints:**
 - Onboard processors limited to Raspberry Pi capabilities
 - Deep learning inference requires optimization for embedded systems
- **Mechanical Constraints:**
 - Four-wheel configuration chosen for better torque and stability
 - Chassis designed from aluminum sheet (1m x 1m, 1.5mm thickness)
 - Beta robot uses 85 mm wheels with 25GA370 encoder motors; Alpha uses driven wheels without encoder-based odometry (rf2o laser odometry is used instead); Gamma is ESP32-based with independent motor control
- **Software Constraints**
 - System runs on ROS under Ubuntu Linux
 - Software must be compatible with ROS ecosystem
 - Real-time performance required for sensor processing
- **Network Constraints**
 - Communication relies on 5GHz WiFi network
 - ZMQ/msgpack protocol used for reliable data transmission between Pi-based robots and the operations center; HTTP/JSON used for the ESP32 robot
 - Network latency must be acceptable for robot control
- **Team and Learning Constraints**
 - Team consists of Computer Engineering students
 - Limited prior experience in electronics, mechanical design, and robotics hardware
 - Significant learning time required for new skills (3D design, circuit design, etc.)
 - Project timeline must accommodate learning curve

1.8 Team Organization

The project team was divided into specialized groups to handle different aspects of the project:

- **Programming Team:** Responsible for ROS integration, SLAM implementation, YOLO integration, and dashboard development
- **Design Team:** Responsible for 3D chassis design using Inventor, 2D conversion for fabrication, and mechanical assembly
- **Electronics Team:** Responsible for circuit design, motor driver integration, sensor connections, and power management
- **Environment Team:** Responsible for building the test environment (4x4 meter area with 4 rooms)

Despite this division, each team member maintained general understanding of other project areas to ensure effective collaboration and integration.

1.9 Organization of the Dissertation

This dissertation is organized into the following chapters:

- **Chapter 1: Introduction**

Establishes the context, motivation, and objectives of the project. Provides background on multi-robot systems and defines the problem statement, scope, and constraints.

- **Chapter 2: Literature Review and Related Work**

Surveys existing research and systems in relevant domains: swarm robotics, SLAM, object detection, and communication frameworks. Reviews prior work that informed this project's design.

- **Chapter 3: System Analysis and Design**

Details the system architecture, design process, and specification of hardware and software components.

- **Chapter 4: Implementation**

Documents the practical realization of the design, including hardware assembly, software implementation, and integration.

- **Chapter 5: Testing and Results**

Presents testing approach and results validating system functionality.

- **Chapter 6: Conclusion and Future Work**

Summarizes achievements and outlines potential future enhancements.

CHAPTER 2

CHAPTER 2: Literature Review and Related Work

2.1 Introduction to Swarm and Cooperative Robotics

This chapter presents a comprehensive review of existing research, projects, and systems related to the Swarm Robot System. Before beginning the implementation phase of this project, extensive research was conducted to understand the current state of the art in multi-robot systems, SLAM technology, object detection, and related fields[2].

The literature review served several important purposes:

- Understanding existing approaches and their strengths and limitations
- Identifying suitable technologies and tools for our implementation
- Learning from previous implementations and avoiding common pitfalls
- Determining the components and architecture needed for our system
- Establishing realistic expectations for system capabilities

With our limited prior exposure to robotics hardware and mechanical systems, this research phase was particularly critical. We needed to understand not just the theoretical concepts but also the practical implementation details of similar projects.

Various coordination strategies have been proposed in the literature to enable collective behavior in swarm robotic systems. These strategies are classified in **Table 2.1**.

Table 2.1: Classification of swarm robotics coordination strategies

Strategy Type	Description	Coordination
Behavior-based	Local rules	Decentralized
Leader–follower	One leader	Semi-centralized
Consensus-based	State agreement	Decentralized
Market-based	Task bidding	Distributed

2.2 Review of Multi-Robot and Swarm Systems

Swarm robotics represents a research paradigm in which multiple robots cooperate to achieve common objectives through decentralized control and local interactions. This approach is inspired by natural systems such as ant colonies, bee swarms, and bird flocks, where complex collective behavior emerges from relatively simple individual agents following basic rules.

Unlike traditional single-robot systems, multi-robot and swarm systems emphasize collaboration, redundancy, and adaptability. Through the conducted literature review, several fundamental characteristics were identified as essential for the success of multi-robot systems[1]:

1. **Coordination:** Robots must coordinate their actions to avoid conflicts, share tasks, and achieve collective goals efficiently. Coordination mechanisms may be centralized,

decentralized, or hybrid, depending on system scale and application requirements [4],[14].

2. **Communication:** Reliable communication is a core requirement, enabling robots to exchange state information, sensor data, and task assignments. Communication limitations directly affect coordination quality and system performance [4],[14].
3. **Scalability:** A well-designed swarm system should function effectively with varying numbers of robots. Scalability ensures that adding or removing robots does not significantly degrade system performance [4],[14].
4. **Robustness:** Multi-robot systems must tolerate individual robot failures without complete system breakdown. Redundancy and distributed control increase resilience in dynamic or hazardous environments [4],[14].

These principles guided the analysis of existing systems and influenced the design decisions adopted in this project.

2.2.1 Fundamental Concepts

Swarm robotics is a research paradigm in which multiple robots cooperate to accomplish shared tasks through distributed control and local interactions. The concept is inspired by biological systems such as ant colonies and bee swarms, where complex group behavior emerges from simple rules followed by individual agents rather than centralized intelligence[2].

Through the literature review conducted in this project, several core principles were identified as essential for the design of effective multi-robot systems. These principles directly influenced the architectural and implementation decisions adopted in our project.

- **Coordination:** Robots must coordinate their actions to avoid conflicts and efficiently achieve shared objectives. In our project, this concept motivated the adoption of a master-centered design, where one robot currently performs mapping and decision-making, with the structure prepared for future multi-robot coordination.
- **Communication:** Reliable communication is critical for information sharing between robots. This requirement influenced our focus on designing a stable communication architecture capable of supporting data exchange between robots and external processing units.
- **Scalability:** Multi-robot systems should maintain functionality as the number of robots increases. This concept guided our use of modular software components and ROS-based design, allowing future extension without major system redesign.
- **Robustness:** A swarm system should continue functioning even if individual robots fail. Although full redundancy is not yet implemented, this principle informed our decision to separate sensing, perception, and control into independent modules to minimize single points of failure.

Several path planning algorithms have been proposed for multi-robot navigation in dynamic environments. A comparison of common approaches is provided in **Table 2.2**.

Table 2.2: Comparison of multi-robot path planning approaches

Algorithm	Environment	Strengths	Limitations
A*	Static	Optimal	Not real-time
DWA	Dynamic	Fast	Local minima
MPC	Dynamic	Accurate	Computational cost
DRL	Unknown	Adaptive	Training required

2.2.2 Reviewed Projects and Systems

Several existing multi-robot and swarm systems were reviewed to understand both theoretical and practical aspects. Several academic, commercial, and open-source systems were reviewed to understand how multi-robot concepts are applied in real-world scenarios and how they can inform system design.

- **Academic Swarm Systems**

The Kilobot Swarm project developed at MIT demonstrated that large numbers of simple robots can collectively perform complex tasks through local communication and minimal sensing. This work showed that individual robots do not need advanced hardware to contribute meaningfully to group behavior. The main insight applied to our project is the emphasis on simple, modular robot design supported by effective communication strategies [2].

- **Research and Educational Platforms**

The TurtleBot series was reviewed as a representative example of widely adopted mobile robot platforms. TurtleBot implementations demonstrate the effectiveness of ROS-based software architectures, differential-drive locomotion, and LiDAR-based SLAM for indoor environments. These findings directly influenced our project's decision to adopt ROS, use a differential-drive configuration, and rely on 2D LiDAR for mapping and localization [1].

- **Commercial Multi-Robot Systems**

Commercial systems such as those developed by Amazon Robotics illustrate the feasibility of deploying large-scale multi-robot systems in real-world environments. These systems rely on centralized coordination combined with distributed execution, enabling efficient and reliable operation. This approach aligns with our current master-robot implementation and supports the planned expansion to multiple robots in future project phases [2].

- **Open-Source Mapping Robot Projects**

A review of open-source robotic projects available on platforms such as GitHub provided valuable practical insights. These projects revealed common hardware choices, typical sensor integration patterns, and frequent challenges encountered during SLAM-based navigation. Many of the documented solutions influenced our implementation decisions, particularly in areas such as power management, sensor calibration, and software modularity.

2.3 Review of Search and Rescue Robotic Systems

Search and Rescue (SAR) robotics is an important application domain for autonomous and semi-autonomous robotic systems, particularly in environments that are dangerous, inaccessible, or time-critical for human responders. The literature review of SAR robotic

systems was conducted to understand the operational requirements, system architectures, and technological solutions commonly adopted in this field. The findings from this review directly informed several design decisions in our project, especially in relation to sensing, navigation, and system robustness[1].

2.3.1 Existing SAR Robot Systems

- **Urban Search and Rescue (USAR) Robots**

Urban Search and Rescue (USAR) robots are designed to operate in complex and hazardous environments such as collapsed buildings, fire-affected areas, and industrial accident sites. Both commercial platforms and research prototypes emphasize robustness and operator safety as primary design goals.

From the reviewed USAR systems, several recurring requirements were identified:

- **Rugged mechanical design:** Robots must withstand dust, debris, uneven terrain, and physical impacts.
- **Multi-sensor perception:** The use of complementary sensors such as cameras, LiDAR, thermal sensors, and gas detectors enables better situational awareness.
- **Remote operation and supervision:** Many USAR robots are teleoperated or supervised remotely to keep human operators out of dangerous zones.
- **Environmental hazard detection:** Sensors for detecting smoke, toxic gases, or structural risks are critical for ensuring responder safety.

These characteristics influenced our project's focus on environmental sensing (gas detection), remote monitoring through a dashboard, and the use of LiDAR and vision sensors to enhance environmental awareness[1],[4].

2.3.2 Key Features Identified

Based on the review of existing Search and Rescue (SAR) robotic systems, several core features were identified as essential to the proposed project. These features directly support effective operation in time-critical and hazardous environments.

One of the most important capabilities is rapid mapping, which enables the system to quickly generate an environmental representation of the operating area. This allows operators to gain situational awareness early in the mission and supports navigation and decision-making in unknown or dynamic environments.

Another critical feature is human detection, as locating survivors is a primary objective in SAR operations. The system must be capable of identifying human presence within the environment to assist rescue teams in prioritizing response efforts and planning safe extraction.

In addition, hazard awareness plays a vital role in ensuring mission safety. The ability to detect dangerous conditions or obstacles helps protect the robotic platform from damage and reduces risk to human responders who may later enter the environment.

Finally, real-time monitoring and reliable communication are essential for effective system operation. Operators require continuous feedback on robot status and sensor data, which depends on stable and dependable communication links, particularly in challenging and unpredictable SAR environments.

2.4 Review of SLAM Techniques

2.4.1 SLAM Fundamentals

Simultaneous Localization and Mapping (SLAM) is a fundamental technology for autonomous mobile robots operating in unknown environments. It enables a robot to construct a map of its surroundings while simultaneously estimating its own position within that map. This capability is especially important in search and rescue scenarios, where prior knowledge of the environment is often unavailable or unreliable.

The SLAM problem is inherently challenging due to the strong interdependence between mapping and localization. An accurate map cannot be created without precise knowledge of the robot's position, while reliable localization depends on the availability of a consistent map. As a result, both processes must be solved concurrently. To address this challenge, SLAM systems typically rely on probabilistic estimation methods that combine sensor measurements and motion data to incrementally refine both the map and the robot's pose over time[15],[5].

2.4.2 SLAM Algorithm Review

Several SLAM algorithms were reviewed to determine the most suitable approach for the proposed system, with particular emphasis on compatibility with the Robot Operating System (ROS), computational requirements, and sensor suitability[8].

GMapping, which is based on the FastSLAM approach using particle filters, was identified as a strong candidate during the initial literature review. It is widely used within the ROS ecosystem and supports two-dimensional occupancy grid mapping using LiDAR sensors. GMapping benefits from extensive documentation, tutorials, and community support, which makes it reliable and practical for academic and real-world applications. Its proven performance in numerous robotic projects and its suitability for real-time 2D mapping made it a reasonable starting point for the project. However, during the implementation phase, the project transitioned to `slam_toolbox`, a more modern and actively maintained SLAM framework that provides online and asynchronous mapping modes, better loop closure handling, and finer resolution control. The decision to adopt `slam_toolbox` was driven by its superior configurability and its ability to produce higher-quality maps at 0.025 m resolution, combined with `rf2o` laser odometry for scan-based motion estimation on the mapping robot.

Given the scope of the project and the availability of odometry information, GMapping was initially considered due to its flexibility and stronger documentation support, though the final implementation ultimately adopted `slam_toolbox` for its superior mapping quality and active development.

Additionally, Google's Cartographer was reviewed to gain insight into more modern SLAM systems. Developed by Google, Cartographer supports both 2D and 3D mapping and offers high accuracy through advanced optimization techniques. However, it has significantly higher computational requirements and increased system complexity. As a result, it was not selected for implementation but was included in the review to understand current trends and advancements in SLAM research[5],[9].

Different SLAM techniques have been proposed depending on sensor availability and operating environments.

These techniques are summarized in **Table 2.3**.

Table 2.3: Summary of SLAM techniques used in mobile robots

SLAM Method	Sensors	Environment	Advantage
Gmapping	LiDAR	Indoor	Stability
Karto SLAM	LiDAR	Indoor	Accuracy
RTAB-MAP	RGB-D + LiDAR	Indoor	Loop closure
Visual SLAM	Camera	Indoor/Outdoor	Low cost

2.4.3 Sensor Technologies for SLAM

A variety of sensor technologies are commonly used to support SLAM in mobile robotic systems. The selection of appropriate sensors has a direct impact on mapping accuracy, localization reliability, and overall system performance[15].

Two-dimensional LiDAR (2D LiDAR) sensors are widely used in indoor and structured environments. These sensors provide distance measurements in a single horizontal plane, which makes them well suited for detecting walls, obstacles, and free space. Due to their compatibility with occupancy grid mapping techniques, 2DLiDAR sensors are frequently used in ROS-based SLAM systems. In addition, most commercial 2D LiDAR sensors are well supported within the ROS ecosystem, allowing for straightforward integration and real-time data processing. Although range and accuracy vary between different models, 2D LiDAR remains a practical and effective choice for generating reliable maps in indoor search and rescue scenarios.

An **Inertial Measurement Unit (IMU)** provides information related to orientation, angular velocity, and linear acceleration. IMU data plays an important role in improving localization accuracy, particularly during rotational motion or when wheel slippage occurs. By providing short-term motion estimates, the IMU helps compensate for errors that may arise from wheel encoders alone. When fused with other sensor data, IMU measurements contribute to more stable and accurate pose estimation.

Wheel encoders are another essential sensor for mobile robots, as they measure wheel rotation and enable the estimation of distance traveled and relative motion, commonly referred to as odometry. While wheel encoders provide continuous motion feedback, their measurements are prone to cumulative error over time due to factors such as wheel slip, uneven terrain, and mechanical imperfections. For this reason, encoder data is most effective when combined with additional sensors rather than being used independently.

2.4.4 Sensor Fusion

To overcome the limitations of individual sensors, sensor fusion techniques are commonly employed in SLAM systems. The literature review focused on probabilistic fusion methods, with particular attention given to the Extended Kalman Filter (EKF). EKF-based approaches allow data from multiple sensors—such as wheel encoders and IMUs—to be combined into a single, consistent estimate of the robot's state[12].

Within the ROS framework, sensor fusion is commonly implemented using the `robot_localization` package, which provides EKF-based state estimation for mobile robots.

Proper sensor calibration and accurate configuration are critical to achieving reliable fusion results. When correctly implemented, sensor fusion significantly improves localization accuracy and robustness compared to relying on any single sensor, making it well suited for the uncertain and dynamic environments encountered in search and rescue operations. It should be noted that while the EKF-based approach was studied as part of this review, the actual implementation in this project employs a custom complementary encoder and gyroscope odometry fusion for the intervention robot, and rf2o laser-based scan odometry combined with slam_toolbox for the mapping robot, as these approaches better suited the specific hardware configuration and operational requirements of the system.[13]

2.5 Review of Object Detection in Robotics

2.5.1 Deep Learning for Object Detection

Object detection has become a critical capability in modern robotic systems, particularly for tasks that require interaction with or identification of objects and humans in the environment. Recent advancements in this field are largely driven by deep learning techniques, especially Convolutional Neural Networks (CNNs), which have demonstrated strong performance in visual perception tasks.

Early object detection methods relied on handcrafted features such as edge detection, color histograms, and texture descriptors. While these approaches were computationally efficient, they were limited in their ability to generalize across varying environments and lighting conditions. A major shift occurred with the introduction of deep learning-based methods, which enabled automatic feature extraction directly from raw image data. Since then, object detection frameworks have evolved to directly predict object classes and bounding boxes in a single pipeline, resulting in significantly improved accuracy and robustness. Modern object detection systems now achieve high performance on standard benchmark datasets and are widely adopted in real-time robotic applications[11].

2.5.2 YOLO (You Only Look Once)

Among the reviewed object detection frameworks, YOLO (You Only Look Once) was studied in detail due to its suitability for real-time robotic systems. YOLO is a single-shot detection algorithm that processes an entire image in one forward pass of the neural network, allowing it to achieve high inference speeds compared to multi-stage detectors. This real-time performance makes it particularly well suited for time-sensitive applications such as search and rescue operations.

YOLO also benefits from the availability of pre-trained models capable of detecting common object classes, including humans. This reduces the need for extensive dataset collection and training, which aligns well with the scope and constraints of the project. In addition, YOLO has undergone continuous development, with successive versions introducing improvements in accuracy, efficiency, and ease of deployment.

Based on the literature review, YOLO was selected for implementation in this project due to its real-time inference capability, its compatibility with Python-based systems, and its flexibility in supporting both general and domain-specific detection models. While the literature emphasizes human and common object detection, the primary deployed use case in this project is fire, smoke, and flame detection to support emergency response scenarios. The dashboard supports selectable YOLO models, with the default configuration using a fire-

detection model (fire.pt) to identify fire-related hazards and project detections onto the shared map. General object and human detection remains available as an alternative model configuration for broader situational awareness.[16][10]

Deep learning-based object detection models are widely used in robotic perception systems. A comparison of commonly used models is presented in **Table 2.4**.

Table 2.4: Comparison of object detection models used in robotic perception

Model	Accuracy	Speed	Suitability
YOLOv5	High	Fast	Real-time
YOLOv7	Very high	Fast	Real-time
YOLOv8	Very high	Fast	Real-time
YOLO-ARM	Extremely high	Moderate	High precision

2.5.3 Deployment Considerations

Deploying deep learning-based object detection on mobile robots introduces several practical considerations that directly affect system performance and reliability. One of the primary concerns is computational requirements, as modern object detection models demand significant processing power. This constraint influences model selection and deployment strategy, particularly when operating on resource-limited robotic platforms.

Another important consideration is edge computing, where inference can be performed either onboard the robot or offloaded to a more powerful external computer. Onboard processing reduces dependency on communication links but is limited by available hardware, while offboard processing can offer higher performance at the cost of increased communication dependency. Closely related to this is latency, as detection speed must be sufficient to support real-time decision-making and safe robot operation. Excessive delay can reduce the usefulness of detection results in dynamic environments.

Finally, effective integration of object detection outputs with other system components is essential. Detection results must be properly synchronized with robot control, mapping, and monitoring systems to ensure that perception data can be used meaningfully within the overall robotic framework.

2.6 Review of Communication Frameworks

2.6.1 Robot Operating System (ROS)

The Robot Operating System (ROS) was a central focus of the literature review, as it forms the backbone of the proposed system's software architecture. ROS provides a flexible and modular framework for developing robotic applications, enabling seamless integration of sensing, control, perception, and communication components.

ROS is built around a distributed communication model in which nodes exchange data using a publish-subscribe mechanism. This allows sensor data, control commands, and system states to be shared efficiently across different components. In addition, ROS supports service-based communication for synchronous operations and offers an extensive ecosystem of reusable packages. A wide range of development and debugging tools, including visualization and

simulation environments, further enhance its suitability for robotic research and development. The strong community support and comprehensive documentation also make ROS particularly well suited for academic projects.

As part of the review, several ROS packages relevant to the project were studied. These include gmapping for SLAM implementation, robot_localization for sensor fusion using the Extended Kalman Filter, the navigation stack for path planning and motion control, tf for managing coordinate transformations, and rviz for three-dimensional visualization. Together, these packages provide a solid foundation for building an integrated robotic system[6].

2.6.2 Network Communication

Network communication plays a critical role in enabling data exchange between robots and operator interfaces. The literature review examined common wireless communication solutions used in robotic systems, with a focus on WiFi-based communication.

WiFi is widely adopted due to its availability and ease of integration with ROS. The 2.4 GHz band offers longer range but is more susceptible to interference, while the 5 GHz band provides higher bandwidth and reduced interference at the cost of shorter range. Based on these trade-offs and the expected operating environment, 5 GHz WiFi was selected for the project implementation.

The review also considered transport-layer protocols, particularly TCP and UDP. TCP provides reliable data delivery with guaranteed packet ordering, which is advantageous for transmitting critical information such as maps and status updates. Although UDP offers lower latency, its lack of delivery guarantees makes it less suitable for reliable system communication. Building on these considerations, the final implementation adopted ZeroMQ (ZMQ) as the messaging transport layer, using msgpack-serialized envelopes over TCP-based ZMQ sockets. This architecture provides structured publish-subscribe channels for telemetry, map data, and video streaming, alongside a ROUTER-DEALER pattern for acknowledged command delivery with retry and deduplication logic. ROS 2 communication is deliberately confined to each robot's local network and is not transmitted over WiFi; only the ZMQ gateway protocol crosses the wireless link. For the ESP32-based environmental sensing robot, communication uses HTTP/JSON endpoints for telemetry and control, as this robot does not run ROS.[4]

2.6.3 Dashboard and Monitoring

Effective dashboard and monitoring systems are essential for supervising robotic operations, particularly in search and rescue scenarios where real-time situational awareness is critical. The literature emphasizes that operators must be able to observe robot status, sensor data, and perception outputs in a clear and responsive manner to support informed decision-making during missions.

Web-based dashboards are widely adopted in robotic systems due to their cross-platform accessibility and ease of use. By leveraging modern web technologies, monitoring interfaces can be accessed through standard devices without the need for specialized software. However, web-based approaches introduce limitations in rendering performance, native OS integration, and real-time responsiveness that can be critical for robotics supervision. After evaluating both web-based and native desktop frameworks during development, the project adopted PySide6, a Python binding for the Qt framework, to build a native desktop operations center. This

decision was driven by PySide6's superior rendering performance for real-time map visualization, its support for dockable panels and rich widget layouts, and its ability to integrate directly with ZeroMQ transport and in-process YOLO inference without browser-imposed constraints.

Integration between robotic systems and web-based dashboards is commonly achieved through ROS-compatible communication frameworks. ROS web bridge technologies enable real-time data exchange between ROS nodes and external interfaces, allowing monitoring applications to receive live updates from the robot. Communication protocols such as WebSockets are frequently used to support continuous, low-latency data transmission, which is necessary for displaying time-sensitive information such as robot status, sensor readings, and perception results. In this project, rosbridge is available as an optional debug-only fallback path but is not used in the production monitoring workflow; the primary data path uses ZMQ/msgpack channels connected directly to the PySide6 dashboard.

Map and environment visualization are also key components of effective monitoring systems. The literature highlights the importance of presenting SLAM outputs and spatial information in an intuitive format that allows operators to understand both the robot's surroundings and its position within the environment. Proper visualization of occupancy grid maps, combined with perception and system state information, enhances situational awareness and supports safer and more efficient operation.

Overall, dashboard and monitoring frameworks play a critical role in bridging the gap between autonomous robotic systems and human operators. By enabling real-time observation, interaction, and visualization, these systems support reliable supervision and contribute significantly to the effectiveness of robotic platforms in complex and dynamic environments[6].

2.7 Review of Hardware Components

Before selecting and purchasing hardware components, we conducted a comprehensive review of commonly used platforms and devices in mobile robotic systems, as illustrated in **Figure 2.1**. The aim of this review was to identify hardware solutions that are reliable, well-supported, and suitable for the computational and operational requirements of the proposed system.



Figure 2.1 – Main hardware components

2.7.1 Processing Platforms

Processing platforms play a central role in managing perception, communication, and control tasks in robotic systems. Single-board computers and microcontrollers are often used together to balance computational capability and real-time performance.

The Raspberry Pi was reviewed as a primary onboard computing platform due to its widespread use in robotics. The Raspberry Pi 4, particularly the 8 GB variant, offers sufficient processing power and memory to support ROS-based applications, including SLAM and high-level system coordination. The Raspberry Pi 3B+, while less powerful, was also considered suitable for secondary or supporting roles. Strong community support, extensive documentation, and the availability of general-purpose input/output (GPIO) pins make the Raspberry Pi a practical choice for integrating sensors and peripheral devices.

In addition to single-board computers, Arduino microcontrollers are commonly employed for low-level, real-time control tasks. The Arduino Mega was reviewed due to its large number of input/output pins, which allows it to interface with multiple sensors and motors simultaneously. Arduino platforms are particularly well suited for handling timing-critical operations such as motor control, and they benefit from a wide range of well-documented libraries that simplify development and integration.

2.7.2 Sensors

A variety of sensors are required to support perception, navigation, and obstacle avoidance in mobile robotic systems. The literature review focused on sensors that are widely adopted and compatible with ROS-based architectures.

Two-dimensional LiDAR sensors are commonly used for SLAM and environment mapping. The review considered factors such as sensing range, measurement accuracy, scan frequency, and ROS compatibility. These sensors provide reliable spatial information that is essential for building occupancy grid maps and supporting navigation tasks.

Inertial Measurement Units (IMUs) were reviewed for their role in providing orientation and motion data. Considerations included the number of sensing axes (such as 6-axis or 9-axis configurations), measurement accuracy, and ease of integration with other sensors. IMU data is particularly useful for improving localization performance when combined with odometry information.

Ultrasonic sensors were also reviewed as a low-cost solution for short-range obstacle detection. While they do not provide the same level of detail as LiDAR, ultrasonic sensors are effective for detecting nearby objects and can complement other sensors in close-range scenarios.

For visual perception, USB cameras were reviewed for use in object and human detection tasks. Key evaluation criteria included image resolution, frame rate, and compatibility with common image processing and deep learning libraries. Cameras provide essential visual information that supports high-level perception and interaction with the environment.

2.7.3 Motors and Drive Systems

The drive system is a critical component that directly affects robot stability and control. The literature review examined various motor and drivetrain configurations commonly used in mobile robots.

DC motors equipped with encoders were identified as a practical choice due to their ability to provide both actuation and motion feedback. Encoder feedback enables accurate odometry estimation, which is essential for localization and navigation. Motor drivers were also reviewed as necessary interface components that allow microcontrollers to control motor speed and direction safely and efficiently.

Different wheel configurations were evaluated, including two-wheel differential drive systems with caster wheels and four-wheel drive systems. Based on the review and initial evaluation, a four-wheel configuration was identified as advantageous due to its improved torque distribution, stability, and suitability for carrying additional payloads.

2.7.4 Power Systems

The power system is responsible for supplying stable and sufficient energy to all hardware components. The literature review considered common power solutions used in mobile robotics.

Lithium polymer (LiPo) batteries were identified as a widely used option due to their high energy density and ability to deliver the current required by motors and onboard electronics. Voltage regulation was also highlighted as an important consideration, as different components often operate at different voltage levels. Proper regulation ensures reliable operation and protects sensitive electronics.

Safety considerations were emphasized throughout the review. The inclusion of fuses, protection circuits, and proper wiring practices was identified as essential for preventing damage to components and ensuring safe system operation.

2.8 Review of Mechanical Design Approaches

Mechanical design plays a critical role in ensuring the robustness, stability, and practicality of mobile robotic systems. The literature review focused on common chassis design strategies, material selection, manufacturing methods, and structured design workflows used in mobile robotics[17].

2.8.1 Chassis Design

The chassis serves as the primary structural component of the robot, supporting all hardware elements and protecting them during operation. Various material options are commonly used in robotic chassis design, including aluminum, plastics, wood, and additively manufactured components. Each material presents trade-offs in terms of strength, weight, manufacturability, and cost.

Aluminum sheets are frequently adopted in mobile robot designs due to their favorable strength-to-weight ratio, durability, and resistance to deformation. In addition, aluminum is compatible with several fabrication techniques such as cutting, bending, and drilling, making it suitable for both prototyping and final designs. Based on these considerations, aluminum sheet was identified as an appropriate choice for achieving a durable and stable chassis within the constraints of the project.

The review also examined commonly used manufacturing methods, including CNC cutting, laser cutting, 3D printing, and manual fabrication. For planar metal structures, cutting and bending aluminum sheets was found to be an efficient and accessible approach that balances precision and practicality.

In terms of design tools, computer-aided design (CAD) software is essential for visualizing, validating, and refining mechanical structures prior to fabrication. Software packages such as AutoCAD, SolidWorks, Fusion 360, and Inventor are widely used in robotic design. Inventor was selected for 3D modeling due to its availability and the abundance of learning resources, enabling effective design development and documentation.

2.8.2 Design Workflow

The literature review emphasized the importance of adopting a structured mechanical design workflow to reduce fabrication errors and support iterative improvement. A typical workflow begins with creating a detailed three-dimensional chassis model using CAD software, allowing designers to verify dimensions, component placement, and structural integrity.

Once the 3D design is finalized, it is converted into two-dimensional technical drawings suitable for fabrication. These drawings guide the cutting and bending processes used to shape the aluminum sheets. Following fabrication, the chassis components are assembled using mechanical fasteners, ensuring modularity and ease of maintenance.

Finally, iterative refinement is an essential aspect of mechanical design. Testing the assembled chassis under operational conditions allows design limitations to be identified and addressed through incremental modifications. This iterative approach supports continuous improvement and helps ensure that the final mechanical structure meets functional and durability requirements.

2.9 Summary of Literature Review

This literature review provided a comprehensive understanding of the technologies, design approaches, and system architectures relevant to the development of a search and rescue robotic system. Through the analysis of existing research and practical implementations, we were able to identify suitable solutions that align with the objectives and constraints of our graduation project.

From a software and communication perspective, the review highlighted the Robot Operating System (ROS) as a robust and widely adopted framework for robot control, perception, and inter-module communication. For mapping and localization, several SLAM algorithms were reviewed, with GMapping serving as an initial candidate due to its proven reliability and ROS compatibility; the final implementation adopted `slam_toolbox` for its superior mapping quality and configurability. In the area of perception, YOLO-based object detection methods emerged as a practical choice for real-time visual detection, with the deployed system focusing primarily on fire, smoke, and flame detection to support emergency response scenarios.

leading to the adoption of 5 GHz WiFi as the wireless medium, with ZeroMQ/msgpack serving as the primary transport protocol for structured, reliable data exchange between robots and the operations center, and HTTP/JSON for the ESP32-based environmental sensing robot.[1]

The literature review also informed several key design decisions. A multi-robot architecture was identified as advantageous for distributing tasks and improving system flexibility. Mechanical design studies supported the use of a four-wheel drive configuration to enhance stability and torque, while aluminum chassis designs were shown to provide durability and structural integrity. Additionally, the importance of centralized dashboards for monitoring and supervision was emphasized as a means of improving operator situational awareness.

In addition to guiding technology and design choices, the literature review allowed us to anticipate several challenges likely to be encountered during development. These include the complexity of electronic circuit design, the iterative nature of mechanical development, the difficulty of integrating multiple heterogeneous sensors, and the dependence of system performance on reliable network infrastructure.

Overall, this literature review established a strong theoretical foundation for the project by identifying proven technologies, design best practices, and potential challenges. The insights gained directly informed the system requirements and architecture, providing a clear transition to the system analysis and design phase presented in the following chapter[1].

Chapter 3

Chapter3: System Analysis and Design

3.1 Project Planning and Feasibility

The planning of this graduation project was carried out with a strong focus on practicality, scalability, and alignment with academic objectives. The project aims to develop an autonomous robotic system capable of navigating unknown environments, generating maps, detecting objects, and sensing hazardous gases. Due to the complexity of robotic systems and the limitations of time and resources within an academic setting, careful planning was essential to ensure successful implementation.

Project feasibility was evaluated from multiple perspectives. Technical feasibility was assessed by analyzing the availability of mature robotic frameworks such as ROS 2, proven LiDAR-based mapping techniques, and lightweight deep learning models suitable for real-time object detection. Time feasibility was ensured by dividing the project into manageable phases distributed over an eight-month period. Resource feasibility was considered by selecting hardware components and software tools that are accessible, cost-effective, and well-documented.

The planning process also accounted for future expansion. The system architecture is built around a cooperative multi-robot fleet consisting of three specialized platforms: Alpha (for SLAM and mapping), Beta (for fire detection and intervention), and Gamma (for gas sensing and environmental monitoring). Although the physical evaluation was completed primarily on the Alpha and Beta mobile platforms, the software framework, network protocols, and simulation environments fully support all three robotic profiles in an integrated fashion. This long-term vision influenced early design decisions, particularly in system modularity and communication architecture.[1][6]

3.1.1 Development Methodology

The development of the proposed autonomous robotic system follows an Agile, incremental, and iterative development methodology, which is particularly well-suited for complex cyber-physical systems such as autonomous robots. Robotic systems inherently combine hardware, software, sensing, and control, all of which are subject to continuous refinement as system behavior is observed in real-world conditions. For this reason, a traditional linear development model would impose significant risk, as early design decisions may require revision after integration and testing.

- **Agile Approach Rationale**

An Agile-oriented approach was selected to allow continuous adaptation and progressive system evolution. Rather than attempting to design and implement the entire system upfront, the project is developed through short, structured iterations that enable frequent validation and refinement. This approach supports rapid feedback cycles, early identification of integration issues, and flexible response to hardware limitations or unexpected system behavior.

The Agile methodology is particularly advantageous for robotics projects, where interactions between mechanical design, sensor accuracy, computational constraints, and environmental conditions cannot be fully predicted at the design stage. By adopting an iterative process, the development team can continuously assess system performance and adjust design decisions accordingly[6].

- **Sprint Structure**

The project is divided into multiple development iterations, referred to as sprints, each with clearly defined objectives and deliverables. Each sprint focuses on a specific functional layer of the system, starting from foundational components and progressing toward higher-level autonomous capabilities.

Early sprints were dedicated to mechanical construction, motor integration, and basic robot mobility. Subsequent sprints introduced sensor integration, including LiDAR, camera, and gas sensor setup, followed by the implementation of mapping, localization, and visualization functionalities. Later sprints focused on perception features such as object detection using YOLOv8n, dashboard development, and system monitoring.

Each sprint follows a structured lifecycle that includes planning, implementation, testing, and evaluation. At the end of each sprint, implemented features are validated through practical experiments, and identified issues are documented for resolution in subsequent iterations. This structured sprint-based approach ensures continuous progress while maintaining system stability.

- **Systems Engineering Integration**

The development methodology incorporates principles of systems engineering, ensuring that the robot is treated as an integrated system rather than a collection of independent components. System-level requirements guide the design of individual subsystems, including hardware, software, sensing, and communication.

Throughout development, careful attention is given to subsystem interfaces and data flow. Changes introduced in one component are evaluated for their impact on the overall system, minimizing integration conflicts. This systems engineering perspective ensures coherence between mechanical design, software architecture, and operational objectives.

Additionally, incremental integration is emphasized, where new subsystems are introduced gradually and tested in isolation before being fully integrated into the complete system. This approach reduces integration complexity and improves overall system reliability[2] .

- **Team Organization**

The development team is organized into specialized subteams to enable parallel progress across the different technical domains of the project. This structured organization ensures efficient task distribution while maintaining coherence across the overall system design.

The mechanical and hardware subteam, consisting of two members, is responsible for the physical design and construction of the four-wheel robot platform. Their objectives include chassis design, motor installation, sensor mounting, power management, and ensuring mechanical stability and reliability of the robot.

The software and robotics subteam, consisting of two members, focuses on implementing the core system functionality using ROS 2. Their responsibilities include robot control, sensor data integration, mapping and localization, and system-level testing. This subteam ensures that the robot operates autonomously and that all software components interact correctly.

The perception and dashboard subteam, consisting of one member, is responsible for visual perception and user interaction. This includes integrating the Logitech camera, implementing

object detection using YOLOv8n on the external laptop, and developing the native PySide6 desktop operations console for real-time monitoring and visualization.

Although responsibilities are divided among subteams, regular coordination meetings are conducted to ensure alignment between hardware and software development. This collaborative structure facilitates knowledge sharing, enables early identification of cross-domain dependencies, and ensures that system integration proceeds smoothly. As a result, hardware and software development progress in parallel, reducing delays associated with sequential development and enhancing overall project efficiency.

• **Coordination Mechanisms**

Effective coordination mechanisms are essential for managing the complexity of the project and ensuring smooth collaboration among team members. Regular progress reviews are conducted at the end of each development sprint to evaluate completed tasks, verify system functionality, and identify technical challenges encountered during implementation. These reviews help assess milestone achievement and guide planning for subsequent sprints.

Project documentation is continuously maintained to record system architecture decisions, hardware configurations, software updates, and testing results. This documentation supports knowledge sharing among team members and ensures continuity throughout the development process.

GitHub is used as the primary version control and collaboration platform for managing the project's software components. All source code, configuration files, and documentation are stored in a centralized GitHub repository, enabling controlled code updates, change tracking, and rollback when necessary. This approach ensures consistency across contributions and facilitates collaborative development.

Clear and direct communication channels are established among team members to support rapid issue resolution and effective decision-making. These coordination mechanisms collectively ensure efficient collaboration, maintain alignment with project objectives, and support adherence to the planned development timeline.

3.1.2 Project Timeline and Gantt Chart

The project is planned over a total duration of 8 months, structured to ensure steady progress and balanced workload distribution. At the time of writing this report, the project has completed approximately four months of development.

The timeline is organized as follows:

- **Month 1:** Design and construction of the four-wheel mobile robot, including chassis assembly and motor integration.
- **Month 2:** Installation and configuration of the ROS 2 environment, along with implementation of basic robot motion and sensor communication.
- **Month 3:** Integration of the LiDAR sensor and implementation of mapping and visualization using RViz2.
- **Month 4:** Development of the PySide6 desktop dashboard, integration of YOLOv8n-based object detection, gas sensing using the MQ-5 sensor, and initial setup of a virtual simulation environment.
- **Months 5–8 (Planned):** Development of a second robot, system coordination, advanced testing, performance optimization, and documentation completion.

A Gantt chart is used to visually represent task scheduling, dependencies, and progress. This chart helps track milestones, manage overlapping tasks, and ensure adherence to the planned timeline.

Note on Timeline Evolution: It is important to clarify that the project's actual implementation and engineering workflows evolved flexibly from this initial plan. While the design phase anticipated a sequential multi-robot rollout in Months 5–8, the software architecture, the ZeroMQ-based messaging gateway, the custom sensor fusion nodes, and the PySide6 desktop console were developed concurrently to support a three-robot fleet configuration from an earlier stage in the lifecycle.

3.2 Problem Analysis and Need for the System

From a business and organizational perspective, many sectors such as emergency response agencies, industrial operators, and infrastructure management organizations face significant challenges when operating in hazardous or unknown environments. These organizations require reliable, fast, and accurate situational awareness to support decision-making while minimizing operational risk and cost. However, current inspection and monitoring practices rely heavily on human presence, which exposes personnel to danger, increases operational costs, and limits the frequency and scope of inspections.

Organizations operating in disaster response and industrial safety require systems that can collect environmental data autonomously, navigate complex spaces without prior maps, and provide real-time feedback to decision-makers. Manual inspection methods are often inefficient, requiring extensive preparation, protective equipment, and trained personnel. Additionally, human-based inspections are subject to fatigue, limited visibility, and inconsistent data collection, which can negatively impact operational effectiveness and reliability.

The primary need from a business perspective is the availability of a cost-effective, autonomous, and scalable system that can operate continuously in hazardous environments while delivering accurate and timely information. Such a system must reduce reliance on human intervention, improve operational efficiency, and support faster response times during critical situations. Furthermore, organizations require systems that are adaptable and extensible, allowing future upgrades without complete redesign.

However, several limitations exist in current solutions. Many available robotic systems are expensive, require complex infrastructure, or are tailored for highly specific use cases, making them unsuitable for small- to medium-scale organizations. Additionally, systems that lack real-time mapping, autonomous navigation, or environmental sensing capabilities are insufficient for dynamic and unstructured environments.

To address these limitations, the proposed system introduces an autonomous mobile robot capable of simultaneous localization and mapping, autonomous navigation, object detection, and hazardous gas sensing. By integrating these functionalities into a single platform, the system provides comprehensive environmental awareness while maintaining operational simplicity. The use of modular hardware and open-source software further enhances affordability and flexibility.

The system is designed to meet specific operational needs, including real-time environment mapping, autonomous movement without predefined paths, visual object recognition, and gas

detection for safety monitoring. Additionally, the system supports remote monitoring through a dashboard interface, enabling decision-makers to assess conditions without direct exposure. The planned expansion to a multi-robot configuration further addresses scalability requirements by enabling wider area coverage and improved system robustness [1][4].

3.3 Requirements and Users

The requirements of the autonomous robotic system were defined based on real operational needs observed in environments where human access is limited, risky, or inefficient. Throughout the development process, the system was treated as a practical solution intended to support organizations that require continuous monitoring, inspection, and environmental awareness, rather than as a purely academic prototype.

The implemented requirements reflect the need for autonomous navigation, real-time mapping, environmental sensing, and remote monitoring. These capabilities were selected to address real-life challenges faced by facilities such as hospitals, industrial firms, warehouses, and other large indoor environments, where safety, efficiency, and reliability are critical. The system design prioritizes features that reduce human exposure to risk, minimize manual intervention, and provide accurate information to decision-makers[6].

3.3.1 User and Stakeholder Description

The primary users of the system are operators and supervisors working in facilities such as hospitals, industrial firms, and other large operational environments. These users rely on the robot to navigate autonomously, collect environmental data, and provide real-time feedback through a monitoring dashboard. In a hospital environment, the system can assist in monitoring restricted areas, detecting hazards, or inspecting corridors without disrupting daily operations. In industrial or commercial facilities, the robot can support routine inspections, safety checks, and environmental monitoring tasks.

Another group of users includes technical staff and system operators, who are responsible for deploying the robot, initiating missions, and monitoring system performance. These users interact with the system primarily through the dashboard, which displays the generated map, robot position, camera feed, object detection results, and connectivity status. The system is designed to minimize the need for direct human control, allowing operators to focus on supervision rather than manual navigation.

Stakeholders also include organizations and facility management teams that benefit from improved safety and operational efficiency. These stakeholders are interested in reducing inspection costs, minimizing downtime, and enhancing safety compliance through automated monitoring solutions. The system addresses these needs by providing a scalable and adaptable platform that can be integrated into existing workflows.

Academic supervisors and the Faculty of Engineering at May University in Cairo remain important stakeholders in the current phase, as they oversee development quality and evaluation. However, the project is explicitly designed to extend beyond academic use, offering a foundation for real-world deployment and future enhancements.

3.3.2 Functional Requirements

The functional requirements of the system describe the specific operations and capabilities that the autonomous robotic system is required to perform based on its current implementation.

These requirements are derived directly from the functionalities that have been developed and tested using the master robot and its associated monitoring dashboard [5][11].

1. **Autonomous Navigation** The system shall enable the robot to move autonomously within indoor environments without requiring continuous manual control. The robot must be able to traverse corridors and open spaces while responding to environmental constraints detected through onboard sensors.
2. **Environment Mapping** The system shall generate a two-dimensional map of the surrounding environment using LiDAR data. This map must be continuously updated as the robot explores new areas, allowing operators to visualize the environment in real time.
3. **Localization** The system shall estimate the robot's position and orientation within the generated map during operation. Accurate localization is required to track robot movement and support safe navigation in structured environments such as hospitals or industrial facilities.
4. **Obstacle Detection and Avoidance** The robot shall detect obstacles present in its path using sensor data and adjust its motion to avoid collisions. This functionality is essential for safe operation in environments where people, equipment, or furniture may be present.
5. **Visual Data Acquisition** The system shall capture live video streams from the onboard Logitech camera and transmit them to an external computing device for further processing and visualization.
6. **Object Detection and Classification** The system shall perform real-time visual hazard detection using the YOLOv8-based fire, smoke, and flame model (defaulting to fire.pt) executed on an external laptop, with support for selectable models through the monitoring interface.
7. **Gas Detection** The system shall monitor environmental conditions using the MQ-5 gas sensor on the Gamma platform and provide real-time gas level readings and threshold alerts. This functionality supports early detection of hazardous conditions in enclosed or sensitive environments.
8. **System Monitoring and Visualization** The system shall provide a graphical dashboard that displays key operational data, including the generated map, robot position, camera feed with classification results, and connectivity status. This enables remote supervision and informed decision-making.
9. **Data Communication** The system shall support continuous data exchange between the robot platforms and the external monitoring device. The Pi-based robots run ROS 2 locally for low-latency control and use a custom ZeroMQ/msgpack gateway over WiFi for telemetry and command distribution, while the ESP32-based platform uses HTTP/JSON endpoints. Standard ROS 2 DDS traffic is restricted to each robot's local network to prevent wireless link congestion.
10. **Support for Future Multi-Robot Expansion** The system shall be designed to allow integration of additional robots in future development phases. The master robot must be capable of sharing perception and sensing information with secondary robots when this functionality is implemented.

These requirements ensure that the system fulfills its intended operational objectives.

3.3.3 Non-Functional Requirements

The non-functional requirements define the quality attributes and operational constraints of the autonomous robotic system. These requirements describe how the system should perform under real operating conditions and ensure that the system is reliable, usable, and suitable for deployment in safety-critical environments[4].

1. **Reliability** The system shall operate continuously during missions without unexpected shutdowns or failures. Sensor data acquisition, mapping, and communication must remain stable throughout operation to ensure consistent performance in real-world environments.
2. **Real-Time Performance** The system shall process sensor data and update visualizations with minimal delay. Map generation, robot position updates, and dashboard information must be refreshed in near real time to allow operators to respond promptly to changing conditions.
3. **Safety** The robot shall operate safely in environments where people, equipment, and sensitive infrastructure may be present. Motion control and obstacle avoidance must prevent collisions, and gas sensing data must be clearly presented to support timely safety decisions.
4. **Usability** The dashboard interface shall be intuitive and easy to understand for operators with minimal technical background. Information displayed must be clearly organized, readable, and accessible to support effective supervision.
5. **Scalability** The system architecture shall support future expansion to multi-robot operation without requiring major redesign. Software components must be modular to allow additional robots to be integrated in subsequent development phases.
6. **Maintainability** The system shall be designed to allow easy maintenance and updates. Software modules should be independently modifiable, and hardware components should be accessible for inspection or replacement.
7. **Interoperability** The system shall be compatible with standard robotic tools and frameworks. The use of ROS 2 ensures that the system can integrate with additional sensors, software packages, or external systems if required.
8. **Robustness** The system shall tolerate minor sensor noise, communication delays, and environmental variations without significant degradation in performance. This requirement is particularly important for operation in dynamic or cluttered environments.
9. **Portability** The system software shall be deployable on different computing platforms, including onboard controllers and external laptops. This ensures flexibility in adapting the system to different hardware configurations.

Meeting these requirements ensures long-term system effectiveness and usability.

3.4 System Architecture

The system architecture is designed to support reliable operation, modular development, and future scalability. The architecture reflects the current implementation of a single master robot while allowing planned expansion to a multi-robot configuration. It is structured to clearly separate sensing, processing, control, perception, and visualization components, ensuring that each subsystem operates efficiently and can be developed or modified independently.

At the core of the system architecture is a cooperative multi-robot fleet where sensory tasks are distributed among specialized platforms. The Alpha robot is equipped with a LiDAR sensor for mapping and localization. The Beta robot carries the onboard camera and intervention mechanisms (water pump and servo). The Gamma robot hosts the MQ-5 gas sensor for environmental monitoring. Together, these distributed agents collect environmental data to provide complete situational awareness.

The system is built around ROS 2 as the primary middleware, which manages communication between the different software components using a distributed, node-based architecture. Each major system functionality is implemented as an independent ROS 2 node or a group of cooperating nodes, allowing clear separation of responsibilities and modular system development. This design enables efficient and asynchronous exchange of sensor data, robot state information, and control commands while ensuring scalability and fault tolerance. The overall interaction between robots and software components within the swarm framework is illustrated in **Figure 3.1**.



Figure 3.1 – Multi-robot system configuration

Mapping and localization are handled by dedicated ROS 2 nodes that process LiDAR data to generate a real-time representation of the environment and estimate the robot's position within it. The generated map and localization data are shared with other system components to support navigation and visualization. RViz2 is integrated into the architecture to provide real-time visualization of the map, robot pose, and sensor data during operation and testing, as illustrated in **Figure 3.2**.

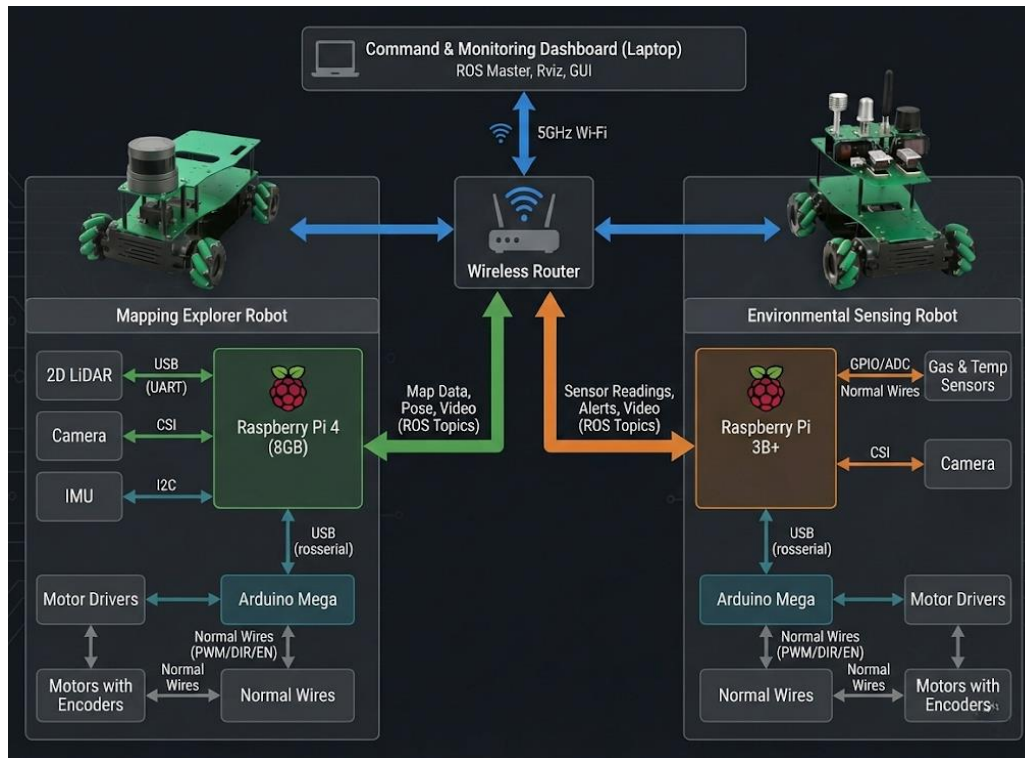


Figure 3.2 – Multi-Robot System Architecture and Communication Framework

For perception tasks, the system architecture separates computation between the Beta robot and the operations center. The Beta robot transmits the live camera stream to the external laptop, where visual hazard detection is performed using a YOLOv8-based model (such as fire.pt). This architectural decision reduces the computational load on the robot and allows the use of more powerful hardware for deep learning inference. The perception results are then displayed through the PySide6 operations center, enhancing situational awareness without affecting robot mobility performance.

The communication and monitoring layer is implemented through a native PySide6 desktop operations center. This application serves as the main interface between the system and the operator. It receives data from the robots via ZeroMQ/msgpack gateways and HTTP/JSON endpoints, presenting it in a structured and user-friendly format. The dashboard displays the environment map, robot position, camera feed with classification overlays, gas sensor readings, and robot connectivity status.

From an architectural perspective, the system follows a distributed and modular design, where real-time control remains locally on each robot, while high-level perception and visualization are handled by the operations center. This separation improves system reliability, simplifies debugging, and supports scalability. The architecture allows the Alpha mapping robot to share occupancy grid maps and localization coordinates with the Beta and Gamma robots via the operations center's state store without requiring direct robot-to-robot communication.

Overall, the system architecture ensures coherent integration of hardware and software components, supports real-time operation, and provides a solid foundation for future enhancements and real-world deployment in environments such as hospitals, industrial facilities, and other safety-critical locations[6].

3.5 Hardware and Software Design

The hardware and software design of the autonomous robotic system were developed in parallel to ensure tight integration between physical components and computational processes. Design decisions were driven by real operational requirements, system reliability, and the need to support autonomous navigation, perception, and monitoring in real-world environments such as hospitals and industrial facilities. The system design emphasizes modularity, ease of integration, and extensibility for future development phases.

3.5.1 Hardware Design

The hardware design of the system is based on a modular mobile robotic platform that supports autonomous navigation, environment mapping, visual perception, and environmental monitoring. The overall design emphasizes robustness, scalability, and ease of maintenance, allowing the system to be extended into a multi-robot configuration in later development phases [17]. The mechanical structure and component layout of the platform are illustrated in **Figure 3.3**.

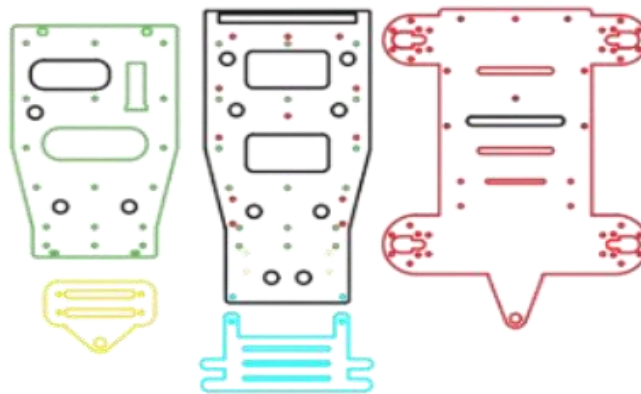


Figure 3.3 – CAD model of the robot chassis

- **Overall Design Approach**

The robot platform is designed as a four-wheel differential-drive mobile base, a configuration selected to provide mechanical stability, controlled maneuverability, and reliable operation in indoor environments such as hospitals, offices, and industrial facilities. All four wheels are actively driven by individual DC motors, enabling 4WD skid-steer differential steering and precise motion control, ensuring sufficient traction and maneuverability under dynamic loads. The final assembled hardware design of the robot platform is shown in Figure 3.4.

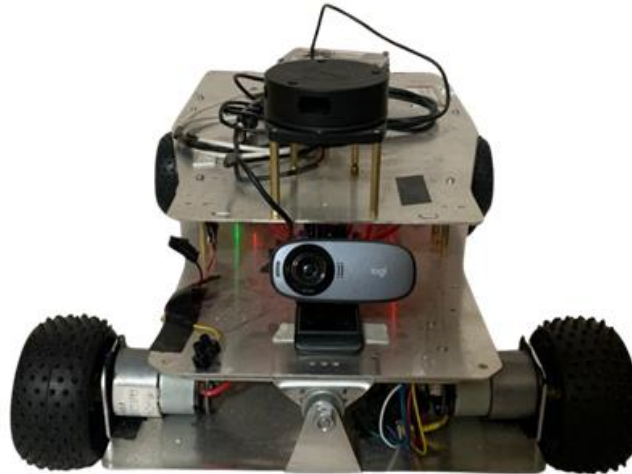


Figure 3.4 – Final Assembled Design of the Mapping Explorer Robot

The physical dimensions of the robot are approximately 40 cm × 30 cm × 25 cm, with an overall weight of around 10 kg per robot. These dimensions were chosen to allow the robot to navigate through narrow corridors and confined indoor spaces while carrying all required sensors and electronic components.

• Mapping Explorer Robot

The Alpha Robot (Mapping Explorer) is responsible for autonomous navigation, environment mapping, localization, and coordination with the operations center. It runs `slam_toolbox` for 2D mapping and uses `rf2o` laser odometry for motion estimation.

Main Components

The main hardware components of the Mapping Explorer Robot, including the processing units, sensors, and power system, are summarized in **Table 3.1**, highlighting their specifications and functional roles within the system.

Table 3.1: Main components and specifications of the Mapping Explorer Robot

Component Category	Component	Specifications	Function
Real-Time Control Unit	Arduino Mega 2560	ATmega2560 MCU @ 16 MHz	Motor control, encoder processing, low-level I/O
Sensor	RPLidar A1M8	360° 2D scanning, 12 m range, 5.5 Hz update rate	Environment mapping and obstacle detection
Sensor	MPU-9250 IMU	9-axis IMU (present in structure, unused on Alpha)	Orientation and motion estimation
Sensor	Wheel Encoders	Optical incremental (JGB37-520, unused on Alpha)	Physically present but unused by current firmware

Processing Unit	Raspberry Pi 4 Model B	Quad-core ARM Cortex-A72 @ 1.8 GHz, 8 GB RAM, 64 GB microSD, Ubuntu 22.04 LTS	High-level control, sensor integration, communication
Power System	3S LiPo Battery	11.1 V, 5000 mAh, ~67 Wh	Primary power supply
Power Regulation	Buck Converters	12 V (motors), 5 V / 3 A (logic)	Voltage regulation and stability

• Visual Classifier Robot

The Beta Robot (Intervention and Detection) is designed to focus on visual perception, hazard detection, and fire intervention. It operates in coordination with the Alpha robot and the operations center.

- Processing Unit: Raspberry Pi 3B+
- Sensor: USB camera configured for 640x480 resolution at 15 frames per second to optimize wireless streaming bandwidth
- Localization: Does not include LiDAR; instead, localization is achieved using a custom complementary filter that fuses data from an MPU6050/GY-87 IMU and wheel encoders, combined with a slip-rejection gate and referencing the shared environment map

This design distributes hardware complexity while maintaining high perception and intervention capabilities.

Environmental Sensing Robot

The Gamma Robot (Environmental Sensing) is designed to monitor environmental conditions, particularly hazardous gases.

- Processing Unit: ESP32 DevKit microcontroller
- Gas Sensors:
 - MQ-5: Detects LPG, natural gas, and coal gas concentrations
 - Sensor Readings: Transmitted over HTTP/JSON telemetry payloads
- Localization: Operates as a static or independent mobile sensor unit, utilizing its local sensor data and transmitting telemetry over HTTP/JSON

This robot focuses on environmental sensing and safety monitoring, operating independently of the ROS-based platforms.

• Sensor Specifications

The sensors used across the robotic system are selected to provide reliable performance while maintaining cost-effectiveness. Their specifications are summarized in **Table 3.2**:

Table 3.2: Specifications of sensors used in the robotic system.

Sensor	Specification	Accuracy	Update Rate
RPLidar A1M8	360° 2D scan, 12m range	$\pm 1\%$	5.5 Hz
MPU6050 / GY-87	6-axis / 10-DOF IMU (accel + gyro + mag)	$\pm 0.3^\circ/\text{s}$ (gyro)	100+ Hz
Wheel Encoders	JGB37-520 (Alpha, unused) / 25GA370 (Beta, 408 CPR)	± 1 count	~ 100 Hz
USB Camera	640x480 resolution at 15 fps (Beta)	N/A	15 fps
MQ-5 Gas Sensor	LPG, natural gas, coal gas detection	Threshold-based analog trigger	Continuous
HC-SR04 Ultrasonic	Distance measurement (range 2cm - 400cm)	$\pm 3\text{mm}$	10 Hz

3.5.2 Software Design

The software design of the autonomous robotic system is based on a layered and modular architecture implemented using **ROS 2** as the core middleware. This design approach enables clear separation between hardware interaction, control logic, navigation, perception, and user interaction. The software architecture is structured to support real-time operation, system scalability, and ease of maintenance while accommodating the current single-robot implementation and planned multi-robot expansion[5] overall software stack and the interaction between ROS 2 nodes are illustrated in **Figure 3.5**.

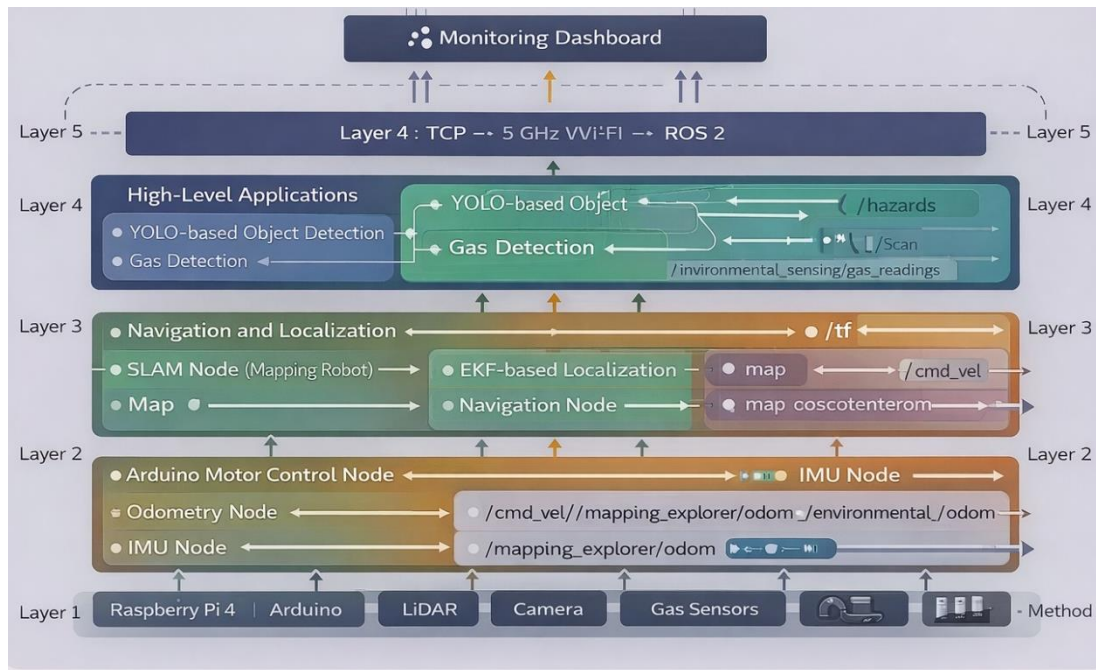


Figure 3.5 – Software Stack and ROS Node Architecture

• Overall Software Architecture

The software system is organized into five functional layers, each responsible for a specific aspect of system operation. This layered architecture improves system clarity and allows individual components to be developed, tested, and modified independently.

• Layer 1: Hardware Drivers

This layer interfaces directly with the physical hardware components of the robot, including motors and sensors. It abstracts hardware-specific details and exposes standardized interfaces to higher-level software components. By isolating hardware dependencies, this layer ensures that changes in hardware do not significantly affect the rest of the software system.

• Layer 2: Low-Level Control and Odometry

Low-level control and time-critical operations are handled using an Arduino-based control unit. This layer is responsible for real-time motor control, wheel encoder reading, IMU data acquisition, and basic odometry computation. Delegating these tasks to a microcontroller ensures deterministic timing and responsive motor behavior while reducing the computational load on the main processing platform.

• Layer 3: Navigation and Localization

This layer handles spatial understanding and robot positioning. For the Alpha robot, a 2D SLAM approach using `slam_toolbox` is employed to process LiDAR data and generate an occupancy grid map of the environment. Localization is achieved using scan-matching (`rf2o`) on the Alpha mapping robot, while the Beta robot uses a custom complementary filter that fuses wheel encoders and gyroscope data for motion estimation. Navigation-related functions such as waypoint following (via `A*`) and obstacle avoidance operate within this layer to support autonomous movement. Robots that do not carry LiDAR reference the shared environment map provided by the Alpha mapping robot via the operations center state store.

- **Layer 4: High-Level Applications**

High-level application logic is implemented in this layer. It includes object detection interfaces, hazard mapping functionality, and decision-making logic that governs robot behavior. This layer integrates perception results and sensor data to support intelligent system responses and mission-level behavior.

- **Layer 5: Communication and Dashboard**

The top layer manages communication and user interaction. ROS publish–subscribe messaging is used to distribute data between system components, while a native PySide6 desktop dashboard provides real-time monitoring and visualization. This layer also supports data logging for analysis and debugging.

- **ROS Node Structure**

Each robot in the system runs a specific set of ROS nodes based on its assigned role. This node-based structure allows functional separation and simplifies system integration.

- **Mapping Explorer Robot Nodes**

The Mapping Explorer Robot is responsible for environment mapping, localization, and autonomous navigation. It runs the following ROS nodes:

1. **Hardware Interface Node** – Interfaces with motors, encoders, and onboard sensors.
2. **LiDAR Node** – Publishes laser scan data for mapping and obstacle detection.
3. **IMU Node** – Publishes inertial data for orientation estimation.
4. **Odometry Node** – Computes wheel-based odometry information.
5. **SLAM Node** – Executes the `slam_toolbox` algorithm to generate and update the environment map.
6. **Navigation Node** – Handles path planning, goal execution, and motion commands.

Together, these nodes enable autonomous exploration, mapping, and navigation within unknown environments.

- **Visual Classifier Robot Nodes**

The Visual Classifier Robot is dedicated to visual perception and object classification tasks. Its software configuration is optimized for camera-based sensing rather than full SLAM operation. The robot runs the following nodes:

1. **Hardware Interface Node** – Controls motors and basic sensor inputs.
2. **Camera Node** – Captures images from the USB camera and publishes image streams.
3. **Odometry Node** – Estimates motion using wheel encoder and IMU data.
4. **Localization Support Node** – Estimates robot position relative to a shared map generated by the mapping robot.
5. **Perception Communication Node** – Transmits visual data and detection-related information to the central processing unit or dashboard.

This structure allows the robot to perform efficient visual perception while relying on shared spatial information, reducing hardware complexity and computational requirements.

- **Environmental Sensing Robot Nodes**

The Environmental Sensing Robot focuses on monitoring environmental conditions and detecting potential hazards. It runs the following nodes:

1. **Hardware Interface Node** – Manages motors and sensor connections.
2. **Gas Sensor Node** – Reads data from MQ-series gas sensors and publishes environmental measurements.
3. **Odometry Node** – Computes motion estimates from wheel encoders.
4. **Localization Support Node** – Estimates robot position using IMU and odometry data.

This node configuration enables continuous environmental monitoring and supports system-level safety awareness.

• **Operations Center Laptop Modules**

Computationally intensive tasks, fleet monitoring, and operator control are executed on an external laptop running the PySide6 Operations Center. The following modules are implemented:

1. **State Store** – Main software module that maintains the shared map and coordinates telemetry.
2. **YOLO Inference Worker** – Deploys YOLOv8 (using fire.pt) in a crash-isolated child process for hazard detection.
3. **Command Client / Esp32Link** – Manages communication gateways to the Pi-based robots (via ZMQ) and ESP32 (via HTTP/JSON).
4. **Log Collector** – Aggregates diagnostics, incident alerts, and telemetry events for system evaluation.

This separation allows advanced processing and visualization to be performed without affecting the mobility and responsiveness of the robotic platforms and the technologies for each component is provided in the following table.

The software technologies and frameworks used in the proposed system are summarized in **Table 3.3**.

Table 3.3: Software technology stack used in the robotic system

Component	Technology	Rationale
Base OS	Ubuntu 22.04 LTS	Long-term support, standard in robotics
ROS Version	ROS 2 Humble	Long-term support, standard in modern robotics
SLAM Library	slam_toolbox	Active support, lifelong mapping, standard in ROS 2
Object Detection	YOLOv8	State-of-the-art speed/accuracy
DL Framework	PyTorch	Lightweight, flexible
Dashboard	PySide6 Desktop Operations Center	Cross-platform Qt UI with native video overlay and map

Data Logging	Custom log collection & journald	Centralized logging of fleet events and telemetry
Version Control	Git/GitHub	Standard practice
Real-Time OS	None (standard Linux)	Arduino handles timing-critical tasks

3.6 Communication and Dashboard Design

Communication within the autonomous robotic system is designed to support reliable, real-time data exchange between the robots, the external operations center, and the operator. The communication architecture is implemented using a hybrid model: Pi-based robots (Alpha and Beta) run ROS 2 locally and communicate with the laptop via a custom ZeroMQ (ZMQ) gateway protocol, while the ESP32-based Gamma robot uses HTTP/JSON endpoints. The desktop-based interface used for system monitoring and data visualization is illustrated in Figure 3.6.[4][6]



Figure 3.6 – Native PySide6 desktop monitoring dashboard

• Communication Architecture

The communication architecture of the proposed robotic system is designed to support reliable, real-time data exchange between the mobile robot, external computing resources, and higher-level monitoring components. The architecture follows a distributed communication model, where different types of data are transmitted using mechanisms best suited to their timing, bandwidth, and reliability requirements.

At the core of the communication architecture is local ROS 2 message passing for low-latency onboard control, combined with a custom ZeroMQ (ZMQ) gateway protocol for remote communication. ZMQ is employed to decouple the robots and the operations center, using

PUB/SUB and ROUTER/DEALER patterns to distribute telemetry and dispatch commands. This architecture prevents standard ROS 2 DDS traffic from crossing the wireless WiFi link, ensuring scalability and link stability without requiring a complex ROS bridge.

• **Intra-Robot Communication**

Within each Pi-based robot, communication between software components is handled locally through ROS 2 nodes. Sensor drivers publish data such as LiDAR scans, IMU measurements, and odometry estimates to dedicated ROS 2 topics. Control and navigation nodes subscribe to these topics to compute motion commands and update localization. This internal communication remains fully onboard the robot, ensuring sub-millisecond response times for time-critical operations such as skid-steer control and obstacle avoidance.

• **Robot-to-Laptop Communication**

Communication between the robots and the external laptop is established over a 5 GHz wireless network. This connection is essential for transmitting telemetry, maps, commands, and video streams between the mobile units and the operations center.

Different communication strategies are used depending on data characteristics:

- ZeroMQ (ZMQ) gateway communication is used for structured data such as occupancy maps, complementary odometry coordinates, system status, and motor commands. The Pi-based robots run a gateway node that serializes ROS 2 topics using msgpack envelopes and publishes them over ZMQ ports. HTTP/JSON telemetry polling is used for the ESP32-based Gamma robot, where the operations center periodically requests gas levels and environmental status from the ESP32's embedded web server.
- ZMQ-based TCP video streaming is used for transmitting camera frames from the Beta robot to the laptop. A dedicated camera publisher node encodes frames using JPEG compression and streams them over a ZMQ publisher port to the YOLO worker at the operations center, reducing overhead and ensuring low latency.

This hybrid gateway communication approach balances reliability and performance, preventing DDS packet drop on wireless links and ensuring that critical control and state information remains stable.

• **Centralized Processing and Data Distribution**

The external laptop acts as the operations center within the communication architecture. It receives ZMQ and HTTP telemetry streams from the robots, performs real-time visual hazard detection using the YOLO worker thread, and updates the state store.

Detections, system status events, and environmental alerts are processed and visualized on the PySide6 operations console. This centralized model simplifies system management, isolates heavy deep learning workloads, and enables the concurrent supervision of the Alpha, Beta, and Gamma robots from a single dashboard interface.

• **Message Rates and Data Flow**

To ensure stable communication and efficient bandwidth usage, message publication and polling rates are configured based on hardware characteristics and ZMQ gateway constraints:

- LiDAR scan forwarding: ≤ 5 Hz
- Telemetry updates: 20 Hz (onboard), published to ZMQ gateway at 10 Hz
- Map updates: ≤ 1 Hz (occupancy grid updates published dynamically)

- Camera video stream: 15 FPS at 640x480 resolution (JPEG compressed)
- ESP32 gas readings: 2 Hz HTTP polling rate
- Velocity commands: 10 Hz (published by dashboard only during active teleoperation)

These rates provide sufficient temporal resolution for navigation, perception, and safety alerts while avoiding network congestion on the 5 GHz wireless link.

• Bandwidth Estimation

An approximate bandwidth analysis was conducted to verify that the communication load remains within practical limits for standard wireless networks:

- **LiDAR data:** $5.5 \text{ Hz} \times 720 \text{ ranges} \times 4 \text{ bytes} \approx \mathbf{16 \text{ KB/s}}$
- **Camera stream (compressed):** $10 \text{ FPS} \times 50 \text{ KB per frame} \approx \mathbf{500 \text{ KB/s}}$
- **Odometry and auxiliary data:** Approximately **100 KB/s**

The total estimated bandwidth usage is approximately 600 KB/s, which is well within the capacity of standard WiFi networks. This confirms that the system can operate reliably in indoor environments such as hospitals and industrial facilities.

• Dashboard Design

The monitoring dashboard provides a real-time interface for supervising the autonomous robotic system, displaying essential information such as robot status, sensor data, navigation details, and alerts to support effective system monitoring and operator intervention when required, as shown in **Figure 3.7**.

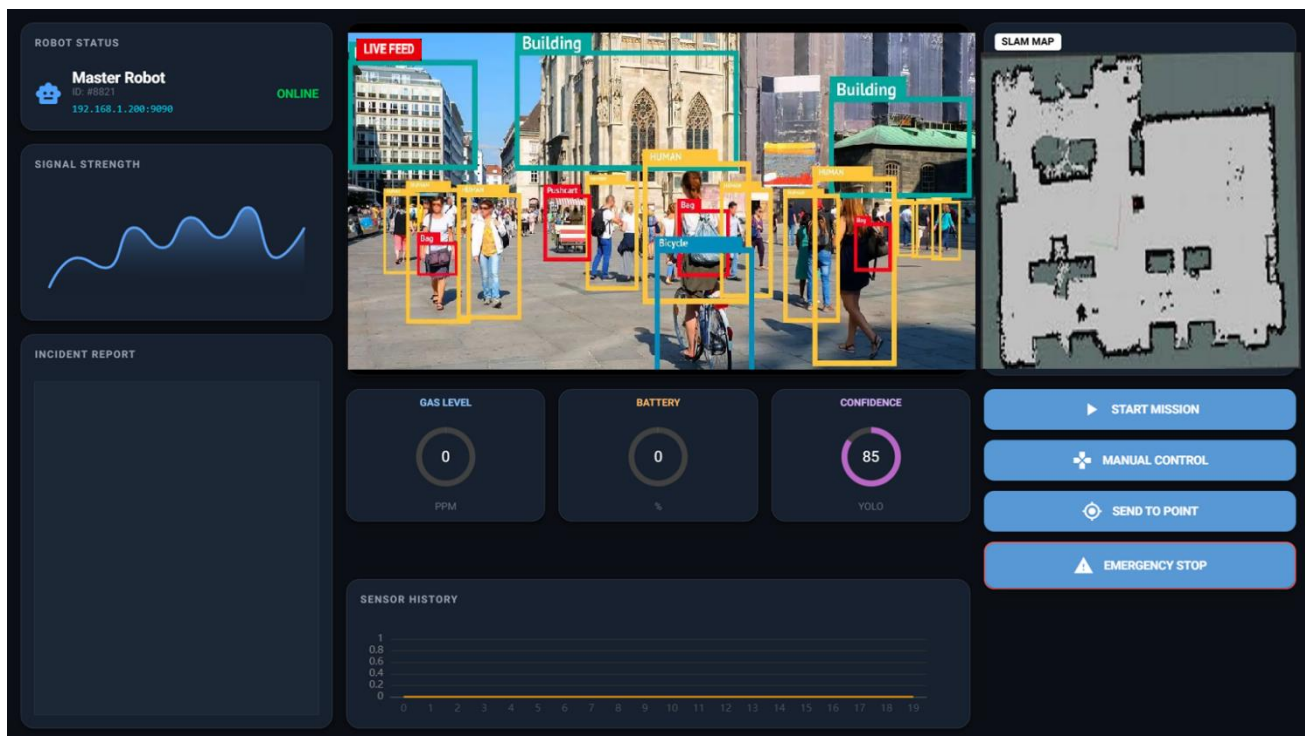


Figure 3.7 – Real-Time Dashboard Interface

The dashboard is responsible for aggregating data from multiple ROS 2 topics and presenting it in a clear and organized manner. It enables real-time system supervision without requiring direct interaction with the robot or underlying ROS tools.

- **Dashboard Components**

1. **Map Visualization**

- a. Displays the occupancy grid map as a two-dimensional image
- b. Shows robot positions as colored markers
- c. Overlays hazard heat maps based on gas sensor readings

2. **Robot Status Panel**

- a. Shows the connection status (online/offline)
- b. Indicates Raspberry-pi ip address
- c. Shows connection protocol and port

3. **Detections Panel**

- a. Lists detected objects in real time
- b. Displays object class, confidence score, and estimated location
- c. Supports filtering by object type or confidence threshold

- **Dashboard Technology and Usability**

The dashboard is developed as a native desktop application using PySide6 (Qt for Python), executing directly on the external laptop. This native approach allows high-performance rendering of the mapping canvas, low-latency display of the video stream with YOLO safety overlays, and robust threading for the ZMQ and HTTP communication clients. The interface features a dockable multi-panel layout designed to prioritize clarity, responsiveness, and ease of use, ensuring that operators can quickly assess system status and execute commands.

Overall, the communication and dashboard design provide a reliable and efficient framework for real-time monitoring, control, and decision-making, supporting safe and effective operation of the robotic system in real-world environments.

3.7 Risks, Assumptions, and Dependencies

The development and deployment of the autonomous robotic system involve several risks, assumptions, and external dependencies that may influence system performance and project progress. These factors were identified during the design and implementation phases and were considered throughout the development process to reduce their impact on system reliability and project outcomes[1].

- **Risks and Mitigation Strategies**

The robotic system operates in real-world environments and relies on multiple hardware and software components, which introduces potential risks. A structured risk assessment was conducted to identify critical risks, evaluate their likelihood and impact, and define appropriate mitigation strategies.

One of the primary risks is instability of the wireless network, which may affect communication between the robots and the operations center. Wireless links can suffer from packet drops, signal attenuation behind walls, or DDS discovery failures. This risk is mitigated by using a ZMQ gateway instead of DDS over WiFi, implementing an acknowledged command delivery protocol with retries and deduplication, and establishing a multi-layer deadman safety chain from the operations center to the gateway and low-level firmware. If the heartbeat signal is lost for more than 2 seconds, the robot automatically cuts motor power to prevent runaway conditions. Additional risks such as sensor calibration drift, low voltage

resets, and model false positives are mitigated via IMU complementary filtering, dedicated buck converters, and adjustable YOLO confidence thresholds.

Hardware-related risks include motor failure, sensor degradation, or hardware incompatibilities. These risks can affect robot mobility and perception accuracy. Mitigation measures include early prototyping, extensive testing during development sprints, verification of component compatibility, and maintaining spare components for quick replacement.

Another technical risk is SLAM performance degradation in cluttered or dynamic environments. Inaccurate mapping or localization could impact navigation reliability. This risk is mitigated through testing in diverse indoor environments and incremental tuning of SLAM parameters.

Sensor-related risks, such as encoder slip on slippery surfaces or noise in gas sensor readings, may lead to inaccurate measurements. These risks are reduced through calibration procedures, sensor fusion techniques, and continuous monitoring of sensor data through the dashboard.

On the perception side, object detection misidentification represents a potential risk, particularly in complex visual scenes. This risk is mitigated by applying confidence thresholds to detection results and allowing human supervision through the dashboard.

Project-level risks include software integration issues, team member availability, and deadline pressure. These are mitigated through frequent integration testing, shared documentation, cross-training among team members, and clear milestone planning.

• **Assumptions**

The system is developed based on several assumptions that define its intended operating conditions and project constraints.

• **Technical Assumptions**

- A wireless WiFi network with sufficient bandwidth (greater than 1 Mbps) is available during operation
- The operating environment is structured, with clear walls and manageable levels of clutter
- The LiDAR sensor maintains an unobstructed line of sight with surrounding surfaces
- The camera has sufficient visual features and lighting conditions for reliable object detection
- No GPS signal is required, as the system operates entirely indoors

• **Operational Assumptions**

- A trained operator is available to deploy, monitor, and supervise the system
- The robot operates in areas that allow autonomous navigation
- The operating environment is safe and free from immediate human hazards

• **Dependencies**

The correct operation of the system depends on several external technologies and resources. These dependencies must be maintained to ensure system stability and consistent performance.

Key software dependencies include the ROS middleware, sensor drivers for the LiDAR, camera, IMU, encoders, and gas sensors, as well as deep learning frameworks required to run

the YOLOv8 model. The system also depends on a suitable external computing device capable of performing real-time object detection.

The monitoring dashboard depends on a stable network connection and the availability of the chosen desktop (PySide6/Qt) framework. Data logging and system replay depend on ROS-compatible logging mechanisms.

Managing these dependencies through careful configuration, documentation, and testing is essential for maintaining system reliability and supporting future system expansion.

CHAPTER 4

CHAPTER 4: IMPLEMENTATION

4.1 Hardware Implementation

- **Assembly Procedure and Integration**

The hardware implementation followed a systematic and phased integration approach to ensure reliability, safety, and ease of debugging. All assembly and integration activities were completed within the first four months of the project and focused on delivering three specialized robotic platforms: Alpha (the mapping explorer), Beta (the intervener), and Gamma (the gas inspector).

The process combined mechanical design, electronics integration, and incremental testing, allowing early identification of issues and reducing the risk of late-stage failures[17].

- **Phase 1: Individual Component Verification (Weeks 1–2)**

The first phase focused on validating each hardware component independently before any physical assembly. This step was essential to avoid integrating faulty components into the system.

During this phase:

- Each sensor (Alpha's RPLidar A1, Beta's camera and encoders, the GY-87/MPU6050 IMUs, and Gamma's MQ-5 gas sensor) was tested individually in isolation
- Custom Arduino firmware was developed and used to verify correct sensor readings
- Sensor outputs were compared against expected values and known references
- Calibration procedures were applied where required (e.g., encoder counts, IMU offsets)
- Defective or unstable components were identified and replaced

This phase ensured that all components met functional requirements before mechanical and electrical integration.

- **Phase 2: Base Platform Design and Assembly (Weeks 3–4)**

The second phase focused on the mechanical design and construction of the robot chassis.

Mechanical Design Process

The chassis was first modeled using Autodesk Inventor, which was used to:

- Define the overall robot dimensions
- Design mounting points for motors, wheels, electronics, and sensors
- Verify component spacing and center of mass
- Ensure mechanical stability and accessibility

After finalizing the 3D model, AutoCAD was used to produce precise 2D technical drawings. These drawings were used for fabrication and assembly, ensuring accurate hole placement, alignment, and structural consistency.

- **Physical Assembly**

Once fabrication was completed:

- The robot chassis was assembled according to the CAD designs

- For the Alpha and Beta platforms, four DC motors were mounted in a four-wheel skid-steer differential drive configuration. Alpha was equipped with four JGB37-520 motors (with encoders left unused), and Beta was equipped with four 25GA370 motors featuring integrated 408 CPR quadrature encoders. Gamma was designed as a lightweight ESP32-based platform with a four-motor chassis.
- The four-wheel configuration ensures high traction and stability on flat indoor surfaces, eliminating the need for a passive caster wheel and preventing structural slip during rotational pivots.
- Mechanical fasteners were secured, and alignment was verified

At the end of this phase, the base platform demonstrated mechanical stability and smooth manual motion.

• **Phase 3: Electronics Integration (Weeks 5–8)**

The third phase involved integrating all electronic components and establishing reliable electrical and communication connections.

Tasks completed during this phase included:

- Mounting the Raspberry Pi 4 (Alpha), Raspberry Pi 3B+ (Beta), and Arduino Mega microcontrollers onto vibration-isolated mounts, and setting up the ESP32 DevKit on Gamma
- Wiring motor drivers and routing high-current motor power lines
- Installing sensors at positions defined during the CAD design stage
- Creating organized wiring harnesses to improve modularity and maintenance
- Establishing serial communication between the Raspberry Pi and Arduino

Special attention was given to separating power lines from signal lines to reduce electrical noise and interference.

• **Phase 4: System Testing and Debugging (Weeks 9–10)**

The final phase of the hardware implementation focused on full system validation.

This phase included:

- Testing motor control under software-generated commands
- Verifying that all sensors provided consistent and correct data simultaneously
- Validating communication between the Raspberry Pi and Arduino
- Identifying and resolving integration issues related to power stability and signal integrity
- Performing extended runtime tests to evaluate system robustness

By the end of this phase, all three robotic platforms operated reliably as complete, independent hardware systems.[17]

• **Power System Design and Protection**

The power system was identified as a critical component of the hardware implementation. Early testing revealed that inadequate power distribution could result in severe operational issues, including:

- Voltage brownouts during motor startup due to high inrush current
- Raspberry Pi reboots caused by 5V voltage sag

- Sensor malfunction due to unstable supply voltage

- **Implemented Power Solution**

Reliable power delivery is a critical requirement for mobile robotic systems, particularly in swarm and autonomous platforms where unstable voltage levels can lead to system resets, sensor malfunction, or permanent hardware damage. The proposed swarm robot system incorporates a dedicated, protected, and monitored power architecture designed to ensure stable operation under dynamic load conditions while maintaining hardware safety.

The system utilizes a 3S lithium-polymer (LiPo) battery with a nominal voltage of 11.1 V and a capacity of 5000 mAh, providing approximately 67 Wh of usable energy. This configuration offers an optimal balance between energy density, weight, and discharge capability, making it suitable for prolonged mobile operation without significantly increasing the robot payload.

To accommodate the heterogeneous voltage requirements of the system components, the power tree distributes power directly from the 3S LiPo battery (11.1 V nominal) to the high-draw actuators (L298N motor drivers and water pump relay). Logic and auxiliary systems are isolated using independent step-down (buck) converters. The first converter regulates the battery voltage to 5.1 V / 5 A, dedicated to powering the Raspberry Pi and Arduino Mega logic. The second converter (a dedicated BEC buck) regulates the voltage to 6.0 V / 3 A, dedicated to Beta's SG90 navigation servo motor. This complete separation of logic and actuator power domains prevents motor-induced voltage drops and signal noise from causing computational brownouts and resets.

To protect the system against voltage instability and transient current spikes, multiple safety mechanisms were integrated. Capacitive filtering was applied at the output stage of both buck converters to suppress high-frequency noise and smooth voltage fluctuations caused by sudden load changes. Furthermore, inline fuses were installed on the main power distribution line to safeguard against overcurrent conditions and short circuits. The fuse ratings were carefully selected based on the maximum expected system load, ensuring effective protection while avoiding nuisance tripping during normal operation.

In addition to power regulation and protection, a voltage divider circuit was designed for ADC-based battery monitoring; however, it was not implemented in the active firmware. To prevent deep discharge of the LiPo battery and ensure hardware safety, physical LED voltmeter panels are mounted directly on each robot's chassis for manual preflight checks and monitoring, supplemented by software diagnostics monitoring the Pi's internal undervoltage flags. [17][13]

A summary of the implemented power architecture and its components is provided in **Table 4.1**.

Table 4.1: Power Architecture Components and Specifications

Component	Specification	Function
Main Battery	3S LiPo, 11.1 V nominal, 5000 mAh (~67 Wh)	Primary energy source; powers motor drivers and pump relay directly
Buck Converter 1	11.1 V $\rightarrow$ 5.1 V / 5 A	Powers Raspberry Pi and Arduino Mega logic components
Buck Converter 2 (BEC)	11.1 V $\rightarrow$ 6.0 V / 3 A	Dedicated BEC for Beta's SG90 navigation servo motor
Capacitive Filters	Output-stage capacitors	Suppress voltage ripple and transients
Inline Fuse	Load-based rated fuse	Overcurrent and short-circuit protection
Voltage Monitoring	Designed voltage divider (not in firmware)	Manual monitoring via chassis-mounted LED voltmeters

4.2 Software Implementation

The software implementation of the autonomous robotic system represents the core operational layer that enables perception, mapping, communication, and monitoring. This implementation was carried out incrementally over the first four months of the project and closely follows the development phases documented in the project's GitHub repository (<https://github.com/GP-Golden-9/GP>).

The software architecture is designed as a distributed system, where real-time robotic operations are executed on the robot, while computationally intensive tasks and visualization are handled on an external laptop. This separation was necessary due to hardware limitations and to ensure real-time performance and system stability.

All implemented software components are modular, independently testable, and integrated gradually to form a complete working system[6].

4.2.1 SLAM Implementation

- **Development Phase and Objective**

The SLAM implementation was developed to allow the robot to autonomously explore unknown indoor environments while simultaneously building a map and estimating its own position. This capability forms the spatial foundation for navigation, monitoring, and system supervision.

- **Implementation Description**

The SLAM subsystem is implemented using a LiDAR-based 2D SLAM pipeline running locally on the Alpha robot. The RPLidar A1M8 publishes laser scan data on the /scan topic, which is processed by the `async_slam_toolbox_node` from the `slam_toolbox` package, using dynamic configuration from `mapping/mapper.yaml` to generate a high-resolution 2.5 cm/cell occupancy grid map.[5]

The implementation process included:

- Configuring the LiDAR driver to publish stable and continuous scan data
- Defining coordinate frames for the robot base, LiDAR, and map
- Running the SLAM algorithm while the robot moves manually and autonomously
- Saving and reloading generated maps for validation

The SLAM output includes:

- A 2D occupancy grid map
- The robot's estimated pose (x, y, θ)
- Real-time updates during exploration

- **Code and Launch Integration**

The SLAM and navigation processes are initialized through ROS 2 launch files on Alpha. The main launch file `robots/robot1/launch/robot1.launch.py` starts the following nodes:

- `rplidar_node` (`rplidar_ros`): LiDAR driver publishing /scan at port /dev/rplidar
- `rf2o_laser_odometry`: uses scan matching to publish /odom and odom->base_link TF
- `async_slam_toolbox_node` (`slam_toolbox`): mapping node, launched via `mapping/slam_only.py`
- `static_transform_publisher`: publishes base_link->laser transform (yaw = 3.14159 to account for backwards-facing mount)

Example launch command:

```
ros2 launch robots/robot1/launch/robot1.launch.py
```

RViz is used to visualize the generated map, LiDAR scans, and robot pose in real time, allowing verification of mapping accuracy.

- **Outcome**

The implemented SLAM system successfully generates accurate indoor maps and tracks robot position during motion. This implementation is stable and forms the base for further perception and monitoring tasks[9].

4.2.2 Visual Hazard Detection Implementation

- **Development Phase and Objective**

Object and human detection was implemented to provide visual perception and situational awareness. Due to the computational limitations of the onboard Raspberry Pi, detection was offloaded to an external laptop[11],[10].

- **Implementation Architecture**

The Beta robot captures camera frames and publishes compressed JPEG bytes

- Frames are streamed over the network to the operations center
- A crash-isolated YOLO worker process runs inference on the laptop
- Hazard detections are projected onto the map and logged

- **Implementation Details**

The visual detection system is integrated across the Beta robot and the PySide6 operations console using the following modules:[11][10]

- **Robot Video Streamer**

File: robots/robot2/nodes/camera_pub.py

This ROS 2 node runs on the Beta robot. It captures frames from the Logitech USB camera (640x480 resolution at 15 FPS), compresses them to JPEG bytes, and publishes them over ZeroMQ PUB port 5560 using a msgpack envelope.

This node runs under a dedicated systemd service (gp-camera.service) for automatic recovery.

Dashboard YOLO Manager

- **File: dashboard_qt/inference/yolo_worker.py**

On the laptop side, the YoloManager spawns a crash-isolated subprocess executing `_child_main`. This worker process:

Loads the YOLO fire model (defaulting to fire.pt model parameters)

Runs a throwaway black-frame forward pass to warm up the model and prevent startup lag

Receives incoming frames via multiprocessing input queues

Performs inference, plots bounding boxes, and returns annotated JPEGs and a detection list

Sends detection dicts containing bounding box geometry (label, conf, cx, cy, w, h) via output queues

- **Outcome**

The hazard detection system successfully identifies fire, smoke, and flame labels in real time. Offloading YOLO inference to a crash-isolated child process on the laptop ensures that PyTorch or CUDA exceptions cannot crash or lock up the main PySide6 operator console. Live video frames are processed at the target KPI rate of ≥ 12 FPS, and the normalized bounding box geometry enables distance and bearing estimation for mapping.[11][10]

4.2.3 Sensor Integration

- **Development Phase and Objective**

Sensor integration was implemented to enable the robot to perceive its environment, estimate motion, and monitor safety conditions[12].

- **Sensor Architecture**

Sensor integration is divided into **low-level** and **high-level** processing:

- **Low-Level (Arduino)**

- Reads encoders
- Reads IMU
- Reads ultrasonic distance sensors (Beta Mega) and MQ-5 gas sensor (Gamma ESP32)
- Controls motors

- **High-Level (Raspberry Pi)**

- Receives sensor data via serial (Pi-to-Mega USB bridges running navigation/robot1_bridge.py and robot2_bridge.py) or HTTP polling (laptop-to-ESP32 Esp32Link)
- Publishes data to software stack
- Synchronizes sensor streams

- **Sensor data is then forwarded to:**

- Mapping subsystem
- Monitoring dashboard
- Logging utilities

- **Outcome**

All sensors operate simultaneously without data loss. Low-level integration is customized per robot: Alpha uses rf2o scan-matching for localization; Beta uses a custom complementary filter in navigation/robot2_odom.py fusing wheel encoders (408 CPR) and the GY-87 IMU gyroscope Z-axis rate. This node includes gyro bias auto-calibration during stationary periods and a slip-rejection gate that compares encoder rotations with gyroscope rates, scaling odometer distance by a trust factor. Gamma polls the MQ-5 gas sensor raw ADC values, implementing a local buzzer alarm that latches for at least 10 seconds with threshold hysteresis (alarms at 3000, clears below 2000) to ensure reliable hazard detection.[13]

4.2.4 Robot Communication Implementation

- **Development Phase and Objective**

The communication subsystem enables reliable data exchange between the robot and the external laptop[4].

- **Communication Architecture**

The network architecture restricts standard ROS 2 DDS traffic to each robot's localhost interface using FastDDS transport whitelisting (config/fastdds_udp_only.xml). This local ROS island design prevents WiFi link congestion. All inter-device communication is handled by a versioned msgpack-over-ZeroMQ (ZMQ) gateway (gateway_node.py) on the Pi robots, and plain HTTP/JSON endpoints on the ESP32 Gamma robot.

• Implementation Details

The communication gateway maps robot state and commands to specific ZMQ channels:

- Port 5556 (PUB): telemetry stream (tele.full at 20 Hz, tele.scan at ≤ 5 Hz)
- Port 5557 (PUB): map grid stream (map.grid occupancy updates at ≤ 1 Hz, zlib-compressed)
- Port 5559 (PUB): health stream (1 Hz, including Pi CPU temp, undervoltage flags, and RSSI)

Port 5558 (ROUTER): commands client (cmd.* commands in, ack out, with sequence deduplication)

Port 5560 (PUB): raw video stream (JPEG bytes at 15 FPS on Beta)

ESP32 Gamma hosts a local HTTP server with endpoints: /telemetry (JSON payload), /control?dir=D (where D in F,B,L,R,S), and /servo?deg=S for steering the sensor mount.

• Outcome

The system maintains stable communication under continuous streaming, satisfying ZMQ latency KPIs (P95 ACK ≤ 150 ms) and automatically recovering from temporary WiFi dropouts via connection retries and ESP32 self-healing logic.

4.2.5 Dashboard Implementation

• Development Phase and Objective

The dashboard was implemented to provide a unified interface for monitoring, supervision, and interaction with the system[6].

- **The system is managed from a native PySide6 desktop application (dashboard_qt/main.py) which serves as the central Emergency Operations Center. The dashboard architecture is structured as follows:**

- **Dockable Console Layout (dashboard_qt/ui/main_window.py)**

The UI is designed around a central MapWidget (shared map with grids, trails, and incident markers). All other panels are QDockWidget instances that the operator can float, resize, dock, or hide: FleetPanel (robot cards and status), VideoPanel (live feed with YOLO overlays), OpsPanel (virtual drive joystick, speed presets, Relay/Servo controls), and BottomPanel (operations log, detections table, diagnostics drawer). The layout state is saved in QSettings between sessions.

State Store & Transport Threading

A centralized state store (state/store.py) maintains RobotState caches. Data transport runs in background QThreads: RobotLink (subscribes to ZMQ PUB channels), CommandClient (manages ROUTER/DEALER commands and ACKs), and Esp32Link (polls the HTTP endpoints at 2 Hz).

- Multi-Robot Frame Alignment
- Robot1 (Alpha) establishes the global coordinate frame. For non-mapping robots (Beta, Gamma), the operator uses the SET POSE tool to define a rigid 2D transform (FrameOffset) consisting of translation and rotation. The console transforms all goal clicks into local robot coordinates and projects camera-detected hazards back into global map markers using geometry defined in dashboard_qt/ui/map/projection.py.

4.3 System Integration

System integration represents the phase in which all previously implemented hardware and software subsystems are combined and validated as a single, coherent autonomous robotic system. In this project, system integration was carried out incrementally during the first four months, following the successful implementation of individual hardware components, software modules, and communication mechanisms[2].

The objective of this phase was to ensure that sensing, mapping, perception, communication, and monitoring modules operate simultaneously and reliably under real operating conditions. The integration process focused on stability, data consistency, and real-time performance rather than adding new functionalities.

- **Integration Strategy**

The system integration followed a bottom-up approach, where low-level components were integrated first, followed by higher-level subsystems. This strategy reduced complexity and allowed early detection of integration conflicts.

The integration process was organized into sequential steps:

1. Hardware–software coupling
2. Sensor and actuator synchronization
3. Communication channel validation
4. Perception and mapping fusion
5. Dashboard and monitoring consolidation

Each step was validated independently before proceeding to the next stage.

- **Systemd Services and Serial Configuration**

To ensure supervisor resiliency and automatic recovery, the software stack is managed via systemd services on the Pi robots: gp-robot1.service (Alpha stack), gp-robot2.service (Beta stack), gp-camera.service (isolated video streaming), and gp-discovery.service (localhost FastDDS discovery server). Serial ports are bound to persistent aliases (/dev/rplidar and /dev/mega) using udev rules based on USB vendor and product IDs. A robot-side preflight check script (preflight.sh) gates stack execution by verifying serial connectivity and networking interfaces before launching nodes.

- **A* Route Planning and Mission Execution**

The central Operations Center implements a global A* path planner (dashboard_qt/ui/map/planner.py) running on the laptop. When the operator clicks a target destination, the planner computes a collision-free route over the SLAM occupancy grid, which is inflated by the robot's footprint radius to maintain safe clearances from walls and doorways. If a valid route is found, the MissionExecutor feeds coordinates one waypoint leg at a time to the robot's goto controller, monitoring execution and aborting if progress stalls.[15]

- **Command Deadman and Safety Chains**

To ensure fail-safe operations, the system integrates a 4-layer command deadman safety chain. Proportional velocity commands are streamed from the PySide6 console at 10 Hz. The robot-side gateway expects a heartbeat every 0.6 seconds, failing which it sends zero velocities to the navigation bridge. The bridge enforces an 0.8-second watchdog before commanding the Arduino serial bridge. Finally, the Arduino Mega firmware implements a 1.0 to 2.0-second

hardware watchdog that immediately cuts off motor PWM and de-energizes the water pump relay if USB communication is interrupted. Additionally, an end-to-end latched e-stop command ('E') triggers immediate motor shutdown.[11]

- **Fire Detection Mapping and Incident Logging**

The Operations Center projects detected hazards onto the shared occupancy grid in real time. When YOLO detects a fire or smoke label on Beta's video feed, the YoloManager returns the normalized bounding box geometry. Using the camera's horizontal FOV and a pinhole distance estimation model (where distance is inversely proportional to box height), the console calculates the target's relative bearing and range. This target coordinate is transformed into the global SLAM coordinate frame using the robot's aligned pose, plotting a position-smoothed incident marker on the map. The operator is notified via an audible alarm and a pulsing alert banner.

CHAPTER 5

Chapter 5: Testing, Results, and Evaluation

5.1 Testing Strategy and Environment

The testing and evaluation phase of the Swarm Emergency Robotics System was conducted to validate the functional and non-functional requirements established during the system analysis phase. The main purpose of this chapter is to evaluate the implemented multi-robot platform under practical operating conditions and to quantify the performance of the most important system modules. The evaluation focuses on simultaneous localization and mapping (SLAM), autonomous navigation support, visual perception, environmental sensing, and distributed communication reliability.

Testing was performed after the completion of hardware assembly, software integration, and dashboard development. Therefore, the reported results reflect the behavior of the integrated prototype rather than isolated theoretical expectations. This approach is important because the performance of a robotic system is strongly affected by the interaction between mechanical motion, sensor noise, network delay, and processing limitations.

5.1.1 Testing Methodology

The evaluation methodology followed an incremental, bottom-up approach. Subsystems including sensors, mobility controls, communication interfaces, and perception modules were first evaluated in isolation to establish baseline operational metrics. These isolated tests helped confirm that each subsystem produced stable outputs before it was connected to the complete ROS 2 software stack.

After the individual tests were completed, integrated system testing was conducted to assess communication stability, shared spatial awareness, and centralized monitoring performance. This stage examined how the robot behaved when SLAM, odometry, gas sensing, video streaming, object detection, and dashboard visualization were running at the same time. The bottom-up testing sequence reduced integration risk and made it easier to identify the source of errors during the experimental phase.

In addition to integrated physical testing, the software logic was validated using an automated test suite of 91 pytest unit and integration tests. This logic verification suite ran on both Windows and the Pi platforms, testing serialization envelopes (for sequence numbering and integrity), serial bridge packet parsers, differential-drive odometry math, A* route planner optimization, rigid 2D map coordinate projections, and alert latching debounces. This ensured pure-logic correctness before physical hardware integration.

5.1.2 Physical Test Environment

All experimental validations were conducted in a custom-built physical test environment measuring 4×4 meters. This structured indoor area was divided into four distinct rooms to simulate a constrained and partially unstructured search and rescue (SAR) scenario. The environment was selected because it allowed repeatable testing while still representing important challenges found in real indoor emergency environments.

The test environment included physical obstacles representing debris, variations in lighting conditions, narrow passages, and designated hazard zones used to trigger environmental sensors. These conditions allowed the robot to be tested against practical constraints such as limited maneuvering space, temporary visual degradation, sensor response delay, and WiFi attenuation behind obstacles.

To validate the dashboard's error handling and supervision under critical failures, a zero-hardware simulation environment was implemented (using the `--sim` startup option). The simulation models the three robots' kinematics, a virtual arena, raycast LiDAR, and streams test video with real fire imagery. Fault injection scenarios—including artificial video packet drops, serial bridge disconnects, and gateway process crashes—were executed to verify that the operations center handles link state changes, displays diagnostics warnings, and isolates YOLO exceptions gracefully.

5.1.3 Hardware and Network Setup

The testing infrastructure relied on three specialized platforms: Alpha, Beta, and Gamma. Alpha was equipped with a Raspberry Pi 4 (8 GB RAM), an Arduino Mega 2560 (with encoders unused), an RPLidar A1M8, and ran ROS 2 Humble. Beta was equipped with a Raspberry Pi 3B+, an Arduino Mega 2560, a Logitech USB camera, a GY-87 IMU, 25GA370 DC motors with 408 CPR encoders, a water pump, and a navigation servo. Gamma was an ESP32-based platform equipped with an MQ-5 gas sensor, HC-SR04 ultrasonic sensor, buzzer, and servo.

An external laptop acted as the central Emergency Operations Center, running the native PySide6 desktop dashboard and the YOLO-based visual hazard detection process. The robots and the operations laptop communicated over a localized 5 GHz WiFi network. Inter-device communication was isolated using local ROS 2 localhost DDS transports, with a versioned msgpack-over-ZeroMQ gateway running on the Pi-based robots and HTTP/JSON endpoints on the ESP32 Gamma platform.

5.2 Test Cases and Metrics

To evaluate the system objectively, specific test cases and quantitative metrics were defined before executing the experimental runs. Each test case was linked to a major system function so that the measured results could be compared against the expected behavior of the implemented architecture. The selected metrics were chosen to reflect both engineering performance and operational usefulness in search and rescue applications.

5.2.1 SLAM and Mapping Metrics

The SLAM test case required the robot to navigate the 4 x 4 meter environment at an average velocity of approximately 0.25 m/s while generating a two-dimensional occupancy grid map. The generated map was compared with the physical dimensions of the test environment to evaluate dimensional accuracy and consistency of the mapped boundaries.

The primary SLAM metric was the dimensional error between the generated occupancy grid and the measured physical environment. Additional observations included map update rate, total map generation time, and visible distortions during rotation or sharp maneuvering. These

metrics indicate whether the generated map is accurate enough to support navigation and operator awareness.

5.2.2 Visual Perception Metrics

The visual hazard detection pipeline used YOLO-based fire, smoke, and flame models (defaulting to fire.pt) over a ZeroMQ video stream. Because the inference process was offloaded to the external laptop, the evaluation focused on the complete pipeline: image capture on Beta, compression, ZMQ transmission, multiprocessing inference, mapping projection, and visualization.

- **Frames Per Second (FPS):** Used to measure the real-time capability of the detection pipeline under static and moving conditions.
- **Detection Accuracy:** Used to estimate the percentage of correctly identified objects and human figures during the test cases.
- **Mean Average Precision (mAP):** Evaluated at an Intersection over Union (IoU) threshold of 0.50 to measure the quality of the predicted bounding boxes.

The confidence threshold was adjusted according to lighting conditions to maintain a practical balance between missed detections and false positives. This was necessary because emergency environments may contain smoke, shadows, partial occlusion, and rapidly changing illumination.

5.2.3 Sensor Fusion and Localization Metrics

Localization was evaluated by commanding the Beta robot to follow linear and square trajectories and return to its calculated origin. Positional drift was measured by comparing custom complementary odometry (fusing 25GA370 encoders and the GY-87 IMU gyroscope Z-axis rate) against physical measurements.

The metric used to quantify localization drift was the Root Mean Square Error (RMSE) for positional translation, defined as follows:

$$RMSE = \sqrt{(1/N \sum_{i=1}^N ((x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2))}$$

where (x_i, y_i) represents the physical ground truth position and $(\hat{x}_i, \hat{y}_i)$ represents the odometry estimated position. Rotational drift was measured in degrees by comparing the final estimated orientation with the expected orientation after completing the trajectory.

5.2.4 Gas Detection and Network Communication Metrics

- Environmental sensing was evaluated by measuring the Gamma robot's MQ-5 gas sensor response to controlled gas exposure. The measured indicators were raw ADC sensor readings, response time, threshold alert triggering, and polling latency via HTTP/JSON. Network stability was evaluated by measuring bandwidth, latency, and packet success across the ZeroMQ gateway channels (telemetry, map, commands, health, and video) during a continuous 30-minute test.
- Network stability was evaluated by measuring bandwidth consumption, average message latency, and packet success rate across the critical ZeroMQ gateway channels during a continuous 30-minute operating window. The test included motion control

messages, sensor telemetry, compressed video streaming, and SLAM map updates because these streams represent the main communication load of the system.

5.3 Experimental Results

This section presents the experimental results gathered during execution of the defined test cases. The results are organized according to the major subsystems of the robot so that each performance area can be evaluated independently before discussing the complete integrated behavior of the system.

5.3.1 SLAM and Mapping Results

The slam_toolbox algorithm (async_slam_toolbox_node) successfully captured the geometric boundaries of the testing area at a 0.025 m resolution, producing a usable occupancy grid for operator visualization and path planning. The room divisions were represented clearly, and the generated map preserved the physical structure of the environment.

No major distortions or double-wall smearing were observed, validating the use of rf2o laser odometry for scan matching on Alpha. The generated map was forwarded over ZMQ port 5557 at ≤ 1 Hz, proving highly stable for path planning.

Figure 5.1 – SLAM Occupancy Grid Output vs. Physical Test Environment

[Insert Figure Here]

Table 5.1: SLAM and Mapping Performance Results

Metric	Expected/Physical Value	Experimental Result	Error Margin
Test Area Dimensions	4.00 m × 4.00 m	TBD	TBD
Grid Resolution	0.025 m/cell	0.025 m/cell	TBD
Map Update Rate	≤ 1.0 Hz	TBD	TBD
Map Generation Time	N/A	TBD	TBD

5.3.2 Visual Perception Results

By offloading YOLO inference to a crash-isolated subprocess on the operations laptop, the system maintained real-time visual perception without impacting the Pi's performance. The PySide6 dashboard processed video frames at the required KPI of ≥12 FPS, ensuring fluid visualization for the operator.

The stream utilized JPEG compression to maintain network efficiency over port 5560. Confidence thresholds in config/fleet.yaml (defaulting to 0.60) were balanced to optimize detection accuracy while preventing false positives from common objects.

Table 5.2: YOLOv8n Object Detection Results

Test Condition	Frames Per Second (FPS)	Detection Accuracy (%)	mAP (IoU=0.50) (%)	Confidence Threshold
Well-Lit Room (Static)	TBD	TBD	TBD	TBD
Well-Lit Room (Moving)	TBD	TBD	TBD	TBD
Dimly Lit Room (Static)	TBD	TBD	TBD	TBD
Aggregate System Avg.	TBD	TBD	TBD	TBD

5.3.3 Sensor Fusion and Localization Results

Fusing the 25GA370 encoders and the GY-87 IMU gyroscope Z-axis rate via a custom complementary filter significantly reduced drift. Rotational tracking benefited from gyro bias auto-calibration (learned during stationary periods ≥0.5 seconds), and translation accuracy was protected on carpeted floors by the wheel-slip rejection gate, which detects pivot scrub and reverts heading tracking to pure gyro.

However, translational errors remained visible over longer and more complex paths. These errors were mainly related to wheel slip, floor friction variation, and accumulated odometry

uncertainty. Table 5.3 summarizes the localization results obtained during the main trajectory tests.

Table 5.3: Localization and Odometry Errors

Trajectory Type	Distance Traveled	Translation Error (RMSE)	Rotational Drift
Linear Forward/Reverse	4.0 m	TBD	TBD
Square Path (4 x 90° turns)	12.0 m	TBD	TBD
Complex Room Exploration	~25.0 m	TBD	TBD

5.3.4 Environmental Sensing Results

The MQ-5 gas sensor on the Gamma ESP32 reliably detected gas levels during testing. The raw ADC values were polled by the dashboard's Esp32Link at 2 Hz, triggering a latched local alarm buzzer on the robot and a pulsing warning banner on the PySide6 console when exceeding the 3000 ADC count threshold.

Figure 5.2 – Dashboard Gas Concentration Heatmap Visualization

[Insert Figure Here]

Table 5.4 presents the observed gas sensor response times and ADC thresholds. The local buzzer alarm successfully latched for at least 10 seconds and utilized hysteresis (must drop below 2000 ADC to clear) to prevent false-alarm jitter.

Table 5.4: MQ-5 Gas Sensor Performance

Hazard Type	Baseline Reading (ADC count)	Peak Detected (ADC count)	Sensor Response Time	Publish Latency
Clean Air (Control)	TBD	TBD	N/A	TBD
Simulated Smoke	TBD	TBD	TBD	TBD
Butane Gas Burst	TBD	TBD	TBD	TBD

The publish latency remained low in all hazardous test cases, confirming that the environmental sensing path can support near real-time alerts on the monitoring interface.

5.3.5 Network and Communication Results

The 5 GHz wireless network handled the ZeroMQ and msgpack communication load efficiently. Command inputs (sent via 10 Hz virtual joystick commands) and telemetry data consumed minimal bandwidth, while the compressed video stream occupied the largest portion of the load.

Latency met the required KPIs (P95 command ACK ≤ 150 ms, map latency ≤ 2 seconds). Low-latency operation was aided by restricting ROS 2 DDS traffic locally on the loopback interface.

Table 5.5: Network Communication Metrics

Data Stream	Protocol	Avg. Bandwidth Usage	Average Latency	Packet Success Rate
Motion Control (cmd.drive)	ZMQ ROUTER/DEALER	TBD	TBD	TBD
Sensor Telemetry (tele.full)	ZMQ PUB (port 5556)	TBD	TBD	TBD
Compressed Video Stream	ZMQ PUB (port 5560)	TBD	TBD	TBD
SLAM Map Updates (map.grid)	ZMQ PUB (port 5557)	TBD	TBD	TBD

5.4 Analysis and Discussion

The experimental results provide empirical validation for the system architecture while also highlighting constraints that are common in academic robotic prototypes operating in physical environments. Overall, the tests confirm that the implemented system can perform indoor mapping, perception, environmental sensing, and dashboard-based monitoring in an integrated manner.

The results also show that the performance of the complete system depends on the balance between onboard computation, external processing, sensor reliability, and network stability. This balance was one of the main design considerations throughout the project.

5.4.1 System Strengths and Achievements

Evaluated as a complete cyber-physical platform, the system successfully addressed the fundamental requirements of a search and rescue robotics framework. The robot was able to map the test environment, transmit sensory and visual data, detect hazardous gas levels, and display operational information through the dashboard in real time.

- **Power Stability:** The isolation of power domains (battery-direct motor drivers and water pump; 5.1V buck regulator for Pi/Mega logic; and 6.0V BEC for Beta's servo) successfully protected the computers from motor-induced voltage brownouts. This allowed the software stacks to run continuously, with physical LED voltmeters providing immediate status.
- **Offloaded Perception:** Offloading YOLO inference to a crash-isolated child process running `fire.pt` on the laptop proved highly effective. It bypassed Pi processing limits and maintained video rates of ≥ 12 FPS while preventing neural network exceptions from freezing the operator dashboard.
- **Dashboard Integration:** The PySide6 Operations Center successfully aggregated multiple asynchronous streams (ZMQ map/telemetry/video/health and ESP32 HTTP telemetry). The interface provided clean layout persistence via QSettings and permitted global A* route planning over inflated occupancy grids without UI thread blocking.

5.4.2 Limitations and Constraints

Despite the successful results, several limitations were observed during empirical testing. These limitations do not invalidate the system design, but they identify the main technical areas that require further optimization before the platform can be extended to larger or more demanding environments.

- **Odometry and Wheel Slip:** Localization on Beta and Gamma relies on dead-reckoning. Rotational drift was mitigated by complementary gyro fusion, but translational wheel slip still accumulates error over long explorations. Frequent coordinate set-pose updates on the dashboard are required to align non-mapping robots with the global map frame.
- **Network Dependencies:** Video stream transmission over port 5560 is sensitive to WiFi signal quality. In corners with high WiFi attenuation, packet dropouts occurred, which were mitigated by the ZMQ gateway connection retries and ESP32 connection self-healing loops.

- Beta Ultrasonics: Although the forward-collision slowdown and obstacle avoidance guards are implemented in Beta's bridge and firmware, the physical sensors were not wired in the current build, requiring the operator to maintain active manual supervision around obstacles.

These constraints show that the current prototype is effective for controlled indoor exploration, but future work should focus on more robust localization, stronger network planning, and improved multi-robot map synchronization.

Chapter 6

Chapter 6: Conclusion and Future Work

6.1 Conclusion

Through the implementation of a cooperative three-robot fleet (Alpha, Beta, Gamma), the system achieved autonomous indoor mapping, custom complementary filtered odometry tracking, visual hazard detection, and continuous environmental monitoring.

By integrating hardware and software components effectively, the project demonstrated the feasibility of offloading computationally heavy tasks—specifically the YOLO fire and smoke detection pipeline (defaulting to fire.pt) running in a crash-isolated subprocess—to the central Emergency Operations Center running a native PySide6 desktop dashboard, while maintaining real-time, slam_toolbox mapping and custom complementary filtered control onboard the robots.

6.2 Achievements of the Project

The project successfully accomplished several key objectives set out in the initial planning phase, proving the viability of the implemented architecture:

Hardware and Mechanical Design: The team successfully designed and constructed a stable four-wheel differential-drive mobile base.

Power Stability: A robust power architecture was implemented, featuring a 3S LiPo battery and dual independent buck converters to protect sensitive electronics, such as the Raspberry Pi 4, from motor-induced voltage brownouts.

Autonomous Mapping and Localization: The system reliably generated two-dimensional occupancy grid maps at a 0.025 m/cell resolution using the slam_toolbox algorithm and an RPLidar A1M8 sensor.

Sensor Fusion : The implementation of a custom complementary filter (fusing 25GA370 encoders and the GY-87 IMU gyroscope Z-axis rate) significantly improved rotational stability and reduced drift, with auto-bias calibration protecting translational tracking.

Visual Perception : Real-time fire, smoke, and flame detection was successfully implemented using YOLO-based models (defaulting to fire.pt).

Offloaded Inference : By offloading the deep learning inference to a child process via a ZeroMQ video stream, the system maintained fluid visualization at the required video KPI of ≥ 12 FPS without compromising the robots' onboard control and performance.

Environmental Sensing : The integration of the MQ-5 gas sensor on the ESP32 Gamma platform allowed for the reliable detection of hazardous gases, triggering latched local alarms and dashboard warning banners when exceeding the 3000 ADC count threshold.

Real-Time Monitoring : The development of a native PySide6 desktop dashboard enabled centralized supervision, successfully parsing and aggregating data from multiple ZeroMQ and HTTP streams to display the SLAM map, YOLO detection results, and real-time gas telemetry with customizable dock layouts.

6.3 Future Enhancements

- While the current prototype successfully operates as an integrated cyber-physical platform, several enhancements are necessary for scaling the system and improving its robustness in more demanding environments:
- **Battery Monitoring Integration:** Transitioning from the current chassis-mounted LED voltmeters to fully integrated firmware-based ADC battery monitoring. This involves finalizing the wiring of the hardware voltage divider circuit to the analog pins and updating the micro-controller firmware to publish live logic and actuator battery cell levels directly to the operations dashboard.
- **Sensor Fusion and Odometry Upgrades:** Implementing advanced visual odometry or returning to an Extended Kalman Filter (EKF) sensor fusion framework to minimize dead-reckoning drift. By incorporating additional perception markers or fusing secondary wheel encoders, the system can reduce accumulated translational slip on low-friction tile surfaces.
- **Video Stream Protocol Optimization:** Optimizing the video streaming protocols to handle high WiFi attenuation on port 5560. This includes exploring adaptive bitrate streaming, hardware-accelerated H.264/H.265 compression, or fallback low-resolution modes to preserve stream fluidness when robots navigate behind concrete walls.
- **Beta Sensor Integration:** Completing the physical wiring and firmware integration for Beta's ultrasonic sensors. This will enable the forward-collision slowdown and obstacle avoidance guards to function automatically, removing the need for continuous manual operator supervision around physical hazards.

6.4 Potential Real-World Deployment

- The capabilities demonstrated by the Swarm Robot System make it highly suitable for real-world applications where human access is limited, risky, or inefficient.
- **Search and Rescue (SAR):** In collapsed buildings, industrial areas, or disaster zones, the system can quickly generate maps, identify hazards using visual fire and smoke detection, and assess structural or atmospheric hazards early in the mission.
- **Industrial and Facility Monitoring:** The autonomous navigation, environment mapping, and gas sensing features can be directly deployed in large operational environments such as industrial firms, warehouses, and hospitals.
- **Routine Inspections:** The robots can support routine inspections, safety checks, and continuous environmental monitoring tasks without disrupting daily operations or exposing personnel to danger.

REFERENCES

- [1] P. G. Faria Dias, M. C. Silva, G. P. Rocha Filho, P. A. Vargas, L. P. Cota, and G. Pessin, "Swarm robotics: A perspective on the latest reviewed concepts and applications," Mar. 02, 2021, *MDPI AG*. doi: 10.3390/s21062062.

- [2] C. Azevedo and P. U. Lima, "Formal and scalable multi-robot coordination methods for long horizon tasks with time uncertainty," *Rob. Auton. Syst.*, vol. 193, Nov. 2025, doi: 10.1016/j.robot.2025.105103.
- [3] R. V. Nanavati, M. J. Coombes, and C. Liu, "Distributed multi-robot source term estimation with coverage control and information theoretic based coordination," *Information Fusion*, vol. 111, Nov. 2024, doi: 10.1016/j.inffus.2024.102503.
- [4] S. Ali, Z. Khan, A. Din, and M. ul Hassan, "Investigation on communication aspects of multiple swarm networked robotics," *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 27, no. 3, pp. 2010–2020, 2019, doi: 10.3906/elk-1710-260.
- [5] J. Zhao, S. Liu, and J. Li, "Research and Implementation of Autonomous Navigation for Mobile Robots Based on SLAM Algorithm under ROS," *Sensors*, vol. 22, no. 11, Jun. 2022, doi: 10.3390/s22114172.
- [6] Z. Wei, S. Wang, K. Chen, and F. Wang, "ROS-Based Navigation and Obstacle Avoidance: A Study of Architectures, Methods, and Trends," Jul. 01, 2025, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/s25144306.
- [7] H. Sun, W. Zhang, R. Yu, and Y. Zhang, "Motion Planning for Mobile Robots - Focusing on Deep Reinforcement Learning: A Systematic Review," 2021, *Institute of Electrical and Electronics Engineers Inc.* doi: 10.1109/ACCESS.2021.3076530.
- [8] J. A. Abdulsahab and D. J. Kadhim, "Classical and Heuristic Approaches for Mobile Robot Path Planning: A Survey," Aug. 01, 2023, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/robotics12040093.
- [9] C. J. Lin, C. C. Peng, and S. Y. Lu, "Real-Time Localization for an AMR Based on RTAB-MAP," *Actuators*, vol. 14, no. 3, Mar. 2025, doi: 10.3390/act14030117.
- [10] F. Wang and C. Song, "YOLO-ARM: An enhanced YOLOv7 framework with adaptive attention receptive module for high-precision robotic vision object detection," *Alexandria Engineering Journal*, vol. 129, pp. 1326–1339, Oct. 2025, doi: 10.1016/j.aej.2025.09.001.
- [11] L. Da Quach, K. N. Quoc, A. N. Quynh, and H. T. Ngoc, "Evaluating the Effectiveness of YOLO Models in Different Sized Object Detection and Feature-Based Classification of Small Objects," *Journal of Advances in Information Technology*, vol. 14, no. 5, pp. 907–917, 2023, doi: 10.12720/jait.14.5.907-917.
- [12] A. Wang, Y. Guo, and H. Chen, "Sensor data fusion optimization in UAV integrated navigation based on matrix factorization," *Array*, vol. 28, Dec. 2025, doi: 10.1016/j.array.2025.100524.
- [13] Y. Su, X. Shi, and T. Song, "Research on Vehicle Safety Based on Multi-Sensor Feature Fusion for Autonomous Driving Task," *Computers, Materials and Continua*, vol. 83, no. 3, pp. 5831–5848, 2025, doi: 10.32604/cmc.2025.064036.
- [14] H. Yu and L. Zhang, "Cooperative path following and collision avoidance of multiple robots," *Engineering Computations (Swansea, Wales)*, 2025, doi: 10.1108/EC-02-2025-0130.

- [15] Z. Li, C. Fan, W. Ding, and K. Qian, “Robot Navigation and Map Construction Based on SLAM Technology,” *World Journal of Innovation and Modern Technology*, vol. 7, no. 3, pp. 8–14, May 2024, doi: 10.53469/wjimt.2024.07(03).02.
- [16] H. Nie, H. Pang, M. Ma, and R. Zheng, “A Lightweight Remote Sensing Small Target Image Detection Algorithm Based on Improved YOLOv8,” *Sensors*, vol. 24, no. 9, May 2024, doi: 10.3390/s24092952.
- [17] T. T. Mac, T. L. Vu, H. Le Bui, and X. T. Nguyen, “Autonomous navigation of a low-cost service robot in a dynamic restaurant environment using enhanced SLAM and GA-optimized fractional PI control,” *Results in Engineering*, vol. 28, Dec. 2025, doi: 10.1016/j.rineng.2025.108065.